PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE
Managed by Jivan Jyot Trust, Amroli.
All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.
Self Financed B.B.A. - B.Com. (Eng. & Guj. Med.) - B.C.A & B.Sc. Data Science Degree Programme
Accredited by National Assessment and Accreditation Council with "B" Grade
Prin. Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Senate Member, VNSGU,Surat
At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)
www.amrolicollege.ac.in
(0261) 2494073, 2495643, 2496478
9427113947, 6354626492
principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

PROJECT REPORT

ON

Analysis of VNSGU Student Academic Performance using Python

AS

A PARTIAL REQUIREMENT FOR THE DEGREE

OF

BACHELOR OF COMPUTER APPLICATION(BCA)

FOR SUBJECT

602 DATA ANALYSIS WITH PYTHON

Submitted by:

Yug Sanjaybhai Patel (6308)

Darshan Mukeshbhai Parmar (6291)

Guided by:

Asst. Prof. Twinkle S. Panchal

Sutex Bank College of Computer Applications and Science, Amroli

Acknowledgement

We would like to express our sincere gratitude to **Asst. Prof. Twinkle S. Panchal**, Sutex Bank College of Computer Applications and Science, Amroli, for her valuable guidance, continuous encouragement, and constant support throughout the development of this project titled **"Analysis of VNSGU Student Academic Performance using Python"**, submitted as a part of the subject **Data Analysis with Python (602)**.

Her expert knowledge, constructive suggestions, and thoughtful feedback helped us understand the concepts of data analysis, exploratory data analysis, visualization techniques, and machine learning implementation more effectively. Her motivation and guidance played an important role in successfully completing this project and improving our understanding of practical data analytics using Python tools and libraries.

We are also thankful to **Sutex Bank College of Computer Applications and Science, Amroli**, for providing us with the necessary academic environment, infrastructure, and learning resources required to carry out this project work successfully. The support from the college helped us gain practical exposure to real-world data analysis techniques and strengthened our knowledge in the field of data science and analytics.

This project provided us with an opportunity to apply theoretical concepts such as data preprocessing, statistical analysis, data visualization, and machine learning clustering techniques on real academic datasets. Through this work, we were able to develop a deeper understanding of how data-driven approaches can help in identifying patterns, trends, and insights in educational data.

Finally, we would like to extend our sincere thanks to everyone who directly or indirectly supported and encouraged us during the completion of this project. Their support and motivation contributed significantly to the successful completion of this work.

Yug Sanjaybhai Patel
Darshan Mukeshbhai Parmar

INDEX

Chapter 1: Project Definition :

Analysis of VNSGU Student Academic Performance using Python

Understanding student academic performance is essential for evaluating educational quality, identifying learning gaps, and improving institutional decision-making. In this project, we analyse the VNSGU student result dataset using Python to extract meaningful academic insights through Exploratory Data Analysis (EDA).

This project focuses on analysing structured examination result data to understand performance distribution, subject-wise trends, grading patterns, and overall academic outcomes.

The primary objective of this analysis is to transform raw examination result data into actionable insights using statistical techniques and data visualization tools.

Project Objectives :

We aim to answer the following analytical questions:

1. What is the overall performance distribution of students?
2. What is the average marks obtained in each subject?
3. Which subject has the highest failure rate?
4. How are total marks distributed among students?
5. What percentage of students passed and failed?
6. Identify top-performing students based on total marks.
7. Are there any outliers in subject-wise marks?
8. What is the correlation between subjects and total marks?

Chapter 2: Dataset Details :

Dataset Name: VNSGU Master Student Result Dataset

Dataset Source:

https://ums.vnsgu.net/Result/StudentResultDisplay.aspx?HtmlURL=7596,0000

Dataset Type: Structured Excel Dataset (.xlsx)

Dataset Domain: Educational Data Analytics / Academic Performance Analysis

Dataset Description

The dataset contains academic performance records of students enrolled under VNSGU. It includes structured information related to student identification, subject-wise marks, total marks, and result status.

The dataset typically contains the following types of attributes:

- Seat Number (Unique Student Identifier)
- Student Name
- Collage Name
- Subject-wise Marks
- Total Marks
- SGPA
- Percentage
- Result Status (Pass/Fail)

Chapter 3: Python tools and libraries used :

Python 3.10 :

Python is a programming language widely used by Data Scientists. Python has in-built mathematical libraries and functions, making it easier to calculate mathematical problems and to perform data analysis.

Pandas :

Pandas is a fundamental, open-source Python library for data manipulation and analysis, providing fast, flexible, and expressive data structures like DataFrames (2D tables) and Series (1D arrays) to handle tabular data efficiently, making it ideal for cleaning, transforming, exploring, and analyzing large datasets in data science, machine learning, and analytics. It simplifies tasks like reading/writing files (CSV, Excel), handling missing data, filtering, grouping, merging, and performing statistical analysis, integrating seamlessly with other tools like NumPy and Matplotlib.

NumPy :

NumPy (Numerical Python) is a foundational Python library for data science, primarily used for its efficient multi-dimensional array object (the ndarray) and high-performance mathematical functions. It is a core component of the Python data science ecosystem, serving as the backbone for other major libraries like Pandas, Matplotlib, and scikit-learn.

Matplotlib :

Matplotlib is a foundational, powerful Python library for creating a wide range of static, animated, and interactive visualizations in data science. It provides extensive customization options and serves as the backbone for many other visualization libraries like Seaborn.

Seaborn :

Seaborn is a powerful Python library for statistical data visualization, built on Matplotlib, that creates attractive, informative plots with minimal code, seamlessly integrating with pandas DataFrames to reveal trends, relationships, and patterns in data, making it essential for Exploratory Data Analysis (EDA). It offers high-level functions for complex plots (like distributions, regressions, heatmaps, violin plots) with beautiful default styles, color palettes, and themes, simplifying the process of understanding data structures and distributions.

Scikit-learn :

Scikit-learn (sklearn) is the most popular, open-source Python library for classical machine learning, providing simple and efficient tools for predictive data analysis. It is built on top of the scientific Python stack (NumPy, SciPy, Matplotlib) and is an essential tool for data scientists.

Chapter 4: Understanding data :

4.1: Importing necessary Python libraries :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

4.2: Creating the data frame :

```
df = pd.read_excel("D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\Data\\raw\\VNSGU_Results.xlsx")
```

4.3 display all column names :

```
print(df.columns)
```

✓ 0.0s

```
Index(['Seat Number', 'Name', 'College Name', 'Total Marks', 'SGPA', 'Result',
       'Course_Name', 'ADVANCE_WEB_TECHNOLOGY_Ext_Theory',
       'ADVANCE_WEB_TECHNOLOGY_Ext_Practical',
       'ADVANCE_WEB_TECHNOLOGY_Total_Ext', 'ADVANCE_WEB_TECHNOLOGY_Int_Theory',
       'ADVANCE_WEB_TECHNOLOGY_Int_Practical',
       'ADVANCE_WEB_TECHNOLOGY_Total_Int', 'ADVANCE_WEB_TECHNOLOGY_Total_Obt',
       'WEB_FRAMEWORK_AND_SERVICES_Ext_Theory',
       'WEB_FRAMEWORK_AND_SERVICES_Ext_Practical',
       'WEB_FRAMEWORK_AND_SERVICES_Total_Ext',
       'WEB_FRAMEWORK_AND_SERVICES_Int_Theory',
       'WEB_FRAMEWORK_AND_SERVICES_Int_Practical',
       'WEB_FRAMEWORK_AND_SERVICES_Total_Int',
       'WEB_FRAMEWORK_AND_SERVICES_Total_Obt', '.NET_TECHNOLOGY_Ext_Theory',
       '.NET_TECHNOLOGY_Ext_Practical', '.NET_TECHNOLOGY_Total_Ext',
       '.NET_TECHNOLOGY_Int_Theory', '.NET_TECHNOLOGY_Int_Practical',
       '.NET_TECHNOLOGY_Total_Int', '.NET_TECHNOLOGY_Total_Obt',
       'LINUX_OPERATING_SYSTEM(LOS)_Ext_Theory',
       'LINUX_OPERATING_SYSTEM(LOS)_Ext_Practical',
       'LINUX_OPERATING_SYSTEM(LOS)_Total_Ext',
       'LINUX_OPERATING_SYSTEM(LOS)_Int_Theory',
       'LINUX_OPERATING_SYSTEM(LOS)_Int_Practical',
       'LINUX_OPERATING_SYSTEM(LOS)_Total_Int',
       'LINUX_OPERATING_SYSTEM(LOS)_Total_Obt', 'NETWORK_TECHNOLOGY_Total_Ext',
       'NETWORK_TECHNOLOGY_Total_Int', 'NETWORK_TECHNOLOGY_Total_Obt',
       'Course_Total_Ext', 'Course_Total_Int', 'Course_Total_Obt'],
      dtype='object')
```

4.5 display top few lines of data :

```
df.head(5)
```

```
    Seat Number                         Name  \
0         1001  ADEP SAIKRISHNA KISHORBHAI
1         1002      AHIR KRISH PRAMODKUMAR
2         1003       AHIR MEET MANSUKHBHAI
3         1004    AHIR MITKUMAR SURESHBHAI
4         1005       AHIR NILAY ARVINDBHAI

                                  College Name Total Marks  SGPA Result  \
0  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   332 / 550  6.18   PASS
1  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   347 / 550  6.36   PASS
2  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   348 / 550  6.18   PASS
3  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   347 / 550  6.36   PASS
4  V.S. PATEL COLLEGE OF ARTS & SCIENCE, BILIMORA   325 / 550  5.82   PASS

                                Course_Name  \
0  CERTIFICATE COURSE ON GOOGLE PRODUCTS(CCGP)
1  CERTIFICATE COURSE ON GOOGLE PRODUCTS(CCGP)
2  CERTIFICATE COURSE ON GOOGLE PRODUCTS(CCGP)
3  CERTIFICATE COURSE ON GOOGLE PRODUCTS(CCGP)
4  CERTIFICATE COURSE ON GOOGLE PRODUCTS(CCGP)
```

```
   ADVANCE_WEB_TECHNOLOGY_Ext_Theory  ADVANCE_WEB_TECHNOLOGY_Ext_Practical  \
0                               11.0                                  22.0
1                                9.0                                  19.0
2                               16.0                                  19.0
...
3              25.0                14              39.0
4              20.0                11              31.0

[5 rows x 41 columns]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

4.6 display shape of data :

```
df.shape
```

```
(8987, 49)
```

4.7 Getting summary of the dataframe using df.info() :

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8987 entries, 0 to 8986
Data columns (total 41 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Seat Number                               8987 non-null   int64
 1   Name                                      8987 non-null   object
 2   College Name                              8987 non-null   object
 3   Total Marks                               8987 non-null   object
 4   SGPA                                      8987 non-null   object
 5   Result                                    8987 non-null   object
 6   Course_Name                               8987 non-null   object
 7   ADVANCE_WEB_TECHNOLOGY_Ext_Theory         8950 non-null   float64
 8   ADVANCE_WEB_TECHNOLOGY_Ext_Practical      8805 non-null   float64
 9   ADVANCE_WEB_TECHNOLOGY_Total_Ext          8793 non-null   float64
 10  ADVANCE_WEB_TECHNOLOGY_Int_Theory         8987 non-null   int64
 11  ADVANCE_WEB_TECHNOLOGY_Int_Practical      8942 non-null   float64
 12  ADVANCE_WEB_TECHNOLOGY_Total_Int          8942 non-null   float64
 13  ADVANCE_WEB_TECHNOLOGY_Total_Obt          8987 non-null   int64
 14  WEB_FRAMEWORK_AND_SERVICES_Ext_Theory     8948 non-null   float64
 15  WEB_FRAMEWORK_AND_SERVICES_Ext_Practical  8830 non-null   float64
 16  WEB_FRAMEWORK_AND_SERVICES_Total_Ext      8819 non-null   float64
 17  WEB_FRAMEWORK_AND_SERVICES_Int_Theory     8986 non-null   float64
 18  WEB_FRAMEWORK_AND_SERVICES_Int_Practical  8940 non-null   float64
 19  WEB_FRAMEWORK_AND_SERVICES_Total_Int      8940 non-null   float64
...
 40  Course_Total_Obt                          8986 non-null   float64
dtypes: float64(31), int64(4), object(6)
```

Chapter 5: Understanding spread of data :

With our dataset we have used histogram with “kde=True” to understand the distribution and spread of data in column “rate” (user ratings).

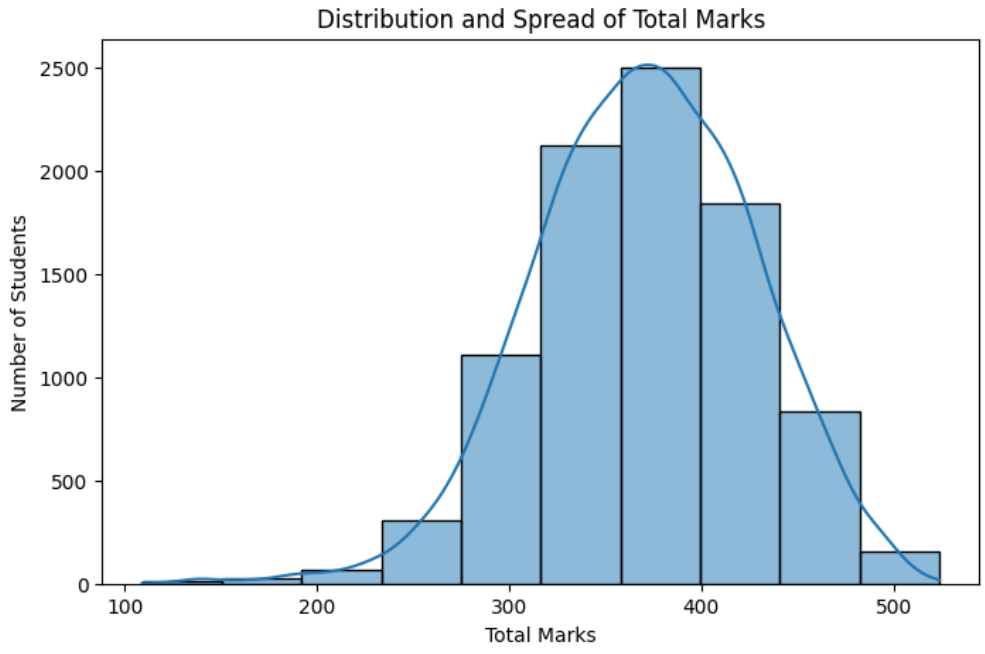
5.1 Distribution of Total Marks (Histogram)

To understand how students performed overall, we visualize the distribution of total marks using a histogram with kernel density estimation (KDE).

A histogram shows:

- How marks are distributed
- Which score range has maximum students
- Whether distribution is normal, skewed, or uneven

```python
plt.figure(figsize=(8,5))
sns.histplot(df['Total'], bins=10, kde=True)
plt.title("Distribution and Spread of Total Marks")
plt.xlabel("Total Marks")
plt.ylabel("Number of Students")
plt.show()
```



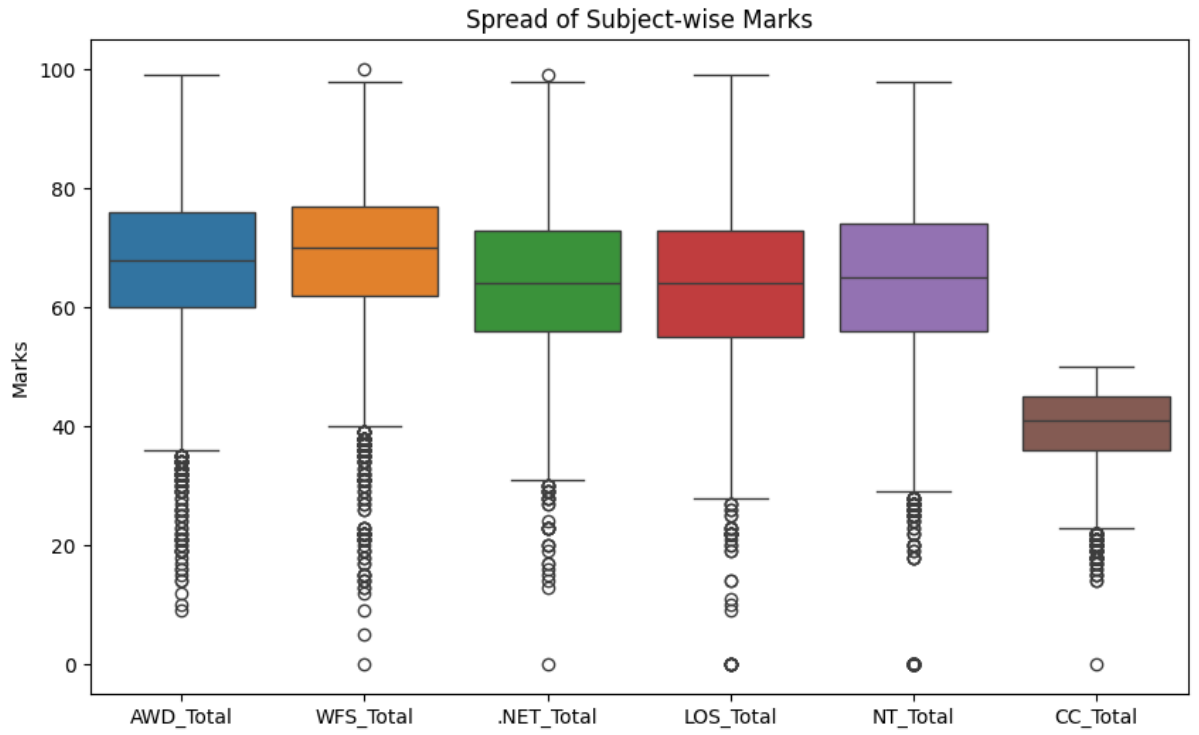
5.2 Spread of Subject-wise Marks (Boxplot Analysis)

To understand variability and detect outliers in individual subjects, we use boxplots.

A boxplot helps in:

- Identifying median marks
- Observing spread (IQR)
- Detecting outliers
- Comparing subject difficulty levels

```python
plt.figure(figsize=(10,6))
sns.boxplot(data=df[['AWD_Total','WFS_Total','NET_Total','LOS_Total','NT_Total','CC_Total']])
plt.title("Spread of Subject-wise Marks")
plt.ylabel("Marks")
plt.show()
```



Chapter 6: Automating EDA using python :

Automating Exploratory Data Analysis (EDA) in Python is achieved using specialized libraries that generate comprehensive, interactive reports with just a few lines of code.

In Python's pandas library, df.info() and df.describe() are fundamental methods for performing initial exploratory data analysis (EDA).

df.info(): Provides a concise structural summary of the DataFrame. It provides following information about dataset:

- Data Types (Dtypes),
- Non-Null Counts,
- Column Names,
- Memory Usage,
- Total Entries/Rows

df.describe(): Offers a statistical summary of the numerical data.

- Count: The number of non-null observations in each column.
- Mean: The average value.
- Std: The standard deviation (spread of data).
- Min: The minimum value.
- 25%, 50%, 75%: The first quartile, median (50th percentile), and third quartile.
- Max: The maximum value.

These functions help data analysts quickly understand the dataset's composition and spot potential issues like missing values or outliers without manually sifting through rows.

6.1: Getting summary of the dataframe using df.info()

```python
print("---------- Dataset Structural Summary ----------")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8987 entries, 0 to 8986
Data columns (total 49 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Seat Number                               8987 non-null   int64
 1   Name                                      8987 non-null   object
 2   College Name                              8987 non-null   object
 3   Total Marks                               8987 non-null   object
 4   SGPA                                      8987 non-null   object
 5   Result                                    8987 non-null   object
 6   Course_Name                               8987 non-null   object
 7   ADVANCE_WEB_TECHNOLOGY_Ext_Theory         8950 non-null   float64
 8   ADVANCE_WEB_TECHNOLOGY_Ext_Practical      8805 non-null   float64
 9   ADVANCE_WEB_TECHNOLOGY_Total_Ext          8793 non-null   float64
 10  ADVANCE_WEB_TECHNOLOGY_Int_Theory         8987 non-null   int64
 11  ADVANCE_WEB_TECHNOLOGY_Int_Practical      8942 non-null   float64
 12  ADVANCE_WEB_TECHNOLOGY_Total_Int          8942 non-null   float64
 13  ADVANCE_WEB_TECHNOLOGY_Total_Obt          8987 non-null   int64
 14  WEB_FRAMEWORK_AND_SERVICES_Ext_Theory     8948 non-null   float64
 15  WEB_FRAMEWORK_AND_SERVICES_Ext_Practical  8830 non-null   float64
 16  WEB_FRAMEWORK_AND_SERVICES_Total_Ext      8819 non-null   float64
 17  WEB_FRAMEWORK_AND_SERVICES_Int_Theory     8986 non-null   float64
 18  WEB_FRAMEWORK_AND_SERVICES_Int_Practical  8940 non-null   float64
 19  WEB_FRAMEWORK_AND_SERVICES_Total_Int      8940 non-null   float64
...
 47  Course_Total_Int                          8987 non-null   int64
 48  Course_Total_Obt                          8986 non-null   float64
```

6.2: Getting description of the dataframe using df.describe()

```python
print("---------- Statistical Summary ----------")
print(df.describe())
```

```
         Seat Number  Total Marks         SGPA   AWD_Ext_Th   AWD_Ext_Pr  \
count   8987.000000  8987.000000  8987.000000  8987.000000  8987.000000
mean    5520.181262   370.020363     6.468580    12.560588    17.758874
std     2609.081077    57.192697     1.689834     4.646001     4.240496
min     1001.000000   109.000000     0.000000     0.000000     0.000000
25%     3264.500000   332.000000     6.000000     9.000000    16.000000
50%     5513.000000   371.000000     6.730000    13.000000    18.000000
75%     7771.500000   410.000000     7.450000    16.000000    20.000000
max    10042.000000   524.000000     9.820000    25.000000    25.000000

       AWD_Ext_Total   AWD_Int_Th   AWD_Int_Pr  AWD_Int_Total    AWD_Total  \
count    8987.000000  8987.000000  8987.000000    8987.000000  8987.000000
mean       30.148103    17.628463    19.849227      37.415266    67.823968
std         7.848625     3.600709     3.735655       6.508946    11.734305
min         0.000000     5.000000     0.000000       0.000000     9.000000
25%        26.000000    15.000000    18.000000      34.000000    60.000000
50%        31.000000    18.000000    20.000000      38.000000    68.000000
75%        35.000000    20.000000    23.000000      42.000000    76.000000
max        49.000000    25.000000    25.000000      50.000000    99.000000

       ...   LOS_Int_Pr  LOS_Int_Total    LOS_Total  NT_Ext_Total  \
count  ...  8987.000000    8987.000000  8987.000000   8987.000000
mean   ...    17.958496      35.452988    63.905530     28.364527
std    ...     4.100120       6.918373    12.854728      9.659826
min    ...     0.000000       0.000000     0.000000      0.000000
...
75%      74.545455
max      95.272727
```

Chapter 7: Handling missing data and outliers (Data cleaning and preparation) :

Before performing deeper analysis, it is essential to clean and prepare the dataset. Raw academic data may contain:

- Missing values
- Incorrect data types
- Duplicate records
- Extreme values (outliers)

Data cleaning ensures accurate and reliable analysis results.

7.1 Handling Missing Values :

Missing values can distort statistical calculations and visualizations. Therefore, we first identify and handle them properly.

```python
print("---------- Checking Missing Values ----------")
print(df.isnull().sum())
```

```
Seat Number                                  0
Name                                         0
College Name                                 0
Total Marks                                  0
SGPA                                         0
Result                                       0
Course_Name                                  0
ADVANCE_WEB_TECHNOLOGY_Ext_Theory           37
ADVANCE_WEB_TECHNOLOGY_Ext_Practical       182
ADVANCE_WEB_TECHNOLOGY_Total_Ext           194
ADVANCE_WEB_TECHNOLOGY_Int_Theory            0
ADVANCE_WEB_TECHNOLOGY_Int_Practical        45
ADVANCE_WEB_TECHNOLOGY_Total_Int            45
ADVANCE_WEB_TECHNOLOGY_Total_Obt             0
```

```
WEB_FRAMEWORK_AND_SERVICES_Ext_Theory          39
WEB_FRAMEWORK_AND_SERVICES_Ext_Practical      157
WEB_FRAMEWORK_AND_SERVICES_Total_Ext          168
WEB_FRAMEWORK_AND_SERVICES_Int_Theory           1
WEB_FRAMEWORK_AND_SERVICES_Int_Practical       47
WEB_FRAMEWORK_AND_SERVICES_Total_Int           47
WEB_FRAMEWORK_AND_SERVICES_Total_Obt            1
.NET_TECHNOLOGY_Ext_Theory                     39
.NET_TECHNOLOGY_Ext_Practical                 103
.NET_TECHNOLOGY_Total_Ext                     113
.NET_TECHNOLOGY_Int_Theory                      1
...
NETWORK_TECHNOLOGY_Total_Obt                   49
Course_Total_Ext                               31
Course_Total_Int                                0
Course_Total_Obt                                1
dtype: int64
```

7.2: Fill Missing Values :

7.2.1Handling Missing Marks :

During data inspection, certain subject columns contained missing values. Upon verification with examination records, these missing values represented students who scored zero marks in specific components (e.g., absent).

```python
# Convert selected columns to numeric first
df[columns_to_fill] = df[columns_to_fill].apply(pd.to_numeric, errors='coerce')

# Then fill NaN with 0
df[columns_to_fill] = df[columns_to_fill].fillna(0)
```

7.2.2 Handling SGPA Column :

The SGPA column contained the value "--" for students who failed the examination. Since this does not represent zero SGPA but rather an undefined grade point, these values were converted to NaN instead of zero to maintain statistical integrity.

```python
df['SGPA'] = df['SGPA'].replace('--', np.nan)
df['SGPA'] = pd.to_numeric(df['SGPA'], errors='coerce')
```

7.2.3 Cleaning Total Marks Column :

The “Total Marks” column contained values in the format “Obtained Marks / Maximum Marks” (e.g., 345/550). Since only the obtained marks were required for analysis, the denominator part was removed.

```python
df['Total Marks'] = df['Total Marks'].astype(str).str.split('/').str[0].str.strip()
df['Total Marks'] = pd.to_numeric(df['Total Marks'], errors='coerce')
```

7.2.4 Converting Marks Columns to Numeric :

The subject mark columns contained some non-numeric values and blank entries. Since these represented either absent students or zero marks, they were converted into numeric format and missing values were replaced with 0.

```python
df[marks_columns] = (
    df[marks_columns]
    .apply(pd.to_numeric, errors='coerce')
    .fillna(0)
    .astype(int)
)
```

Chapter 8 Univariate Analysis :

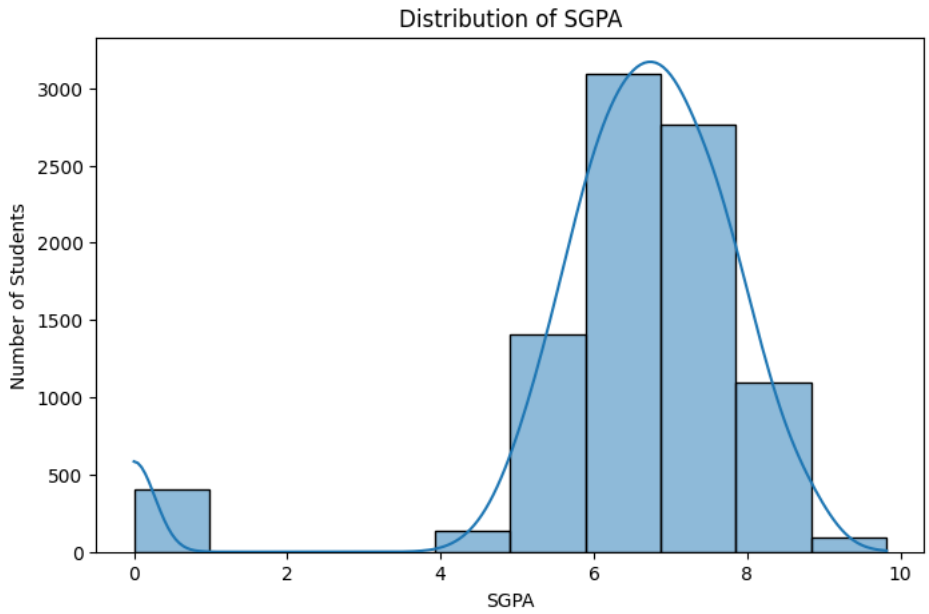
Univariate Analysis involves analyzing a single variable at a time to understand its distribution, central tendency, and frequency patterns.

In this chapter, we analyze:

- SGPA Distribution
- Pass vs Fail Ratio
- Subject-wise Failure Count
- Percentage Distributation

8.1 SGPA Distribution :

```python
plt.figure(figsize=(8,5))
sns.histplot(df['SGPA'], bins=10, kde=True)
plt.title("Distribution of SGPA")
plt.xlabel("SGPA")
plt.ylabel("Number of Students")
plt.show()
```

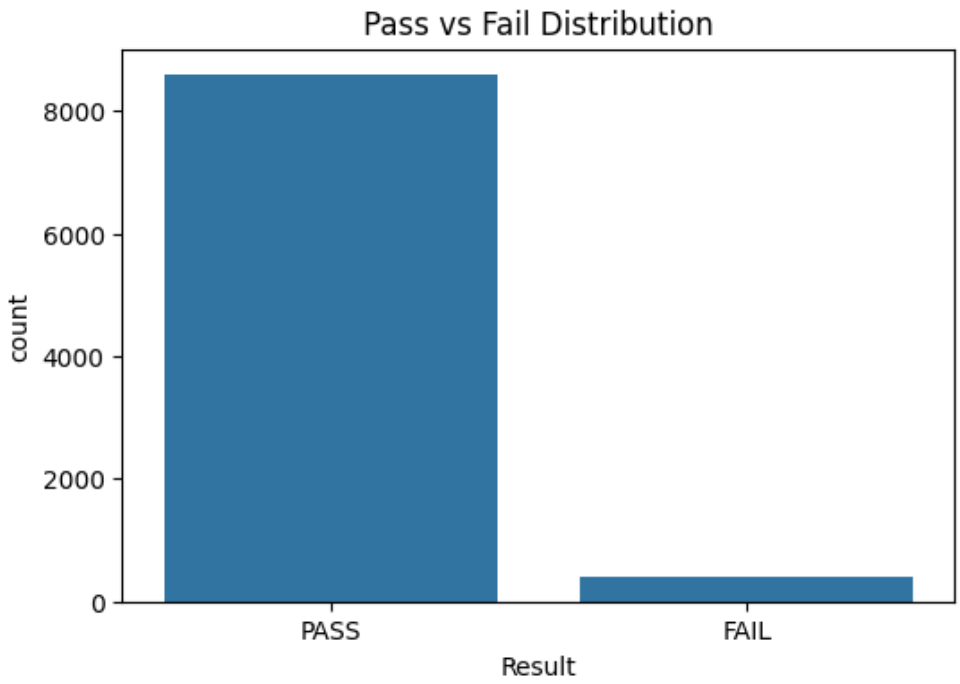


Interpretation:

- Majority clustered around 6–8 average performance.
- High peak near 8–9 strong academic performance.
- Values near 0 indicate failed students (if included).

8.2 Pass vs Fail Distribution :

```python
plt.figure(figsize=(6,4))
sns.countplot(x='Result', data=df)
plt.title("Pass vs Fail Distribution")
plt.show()
```



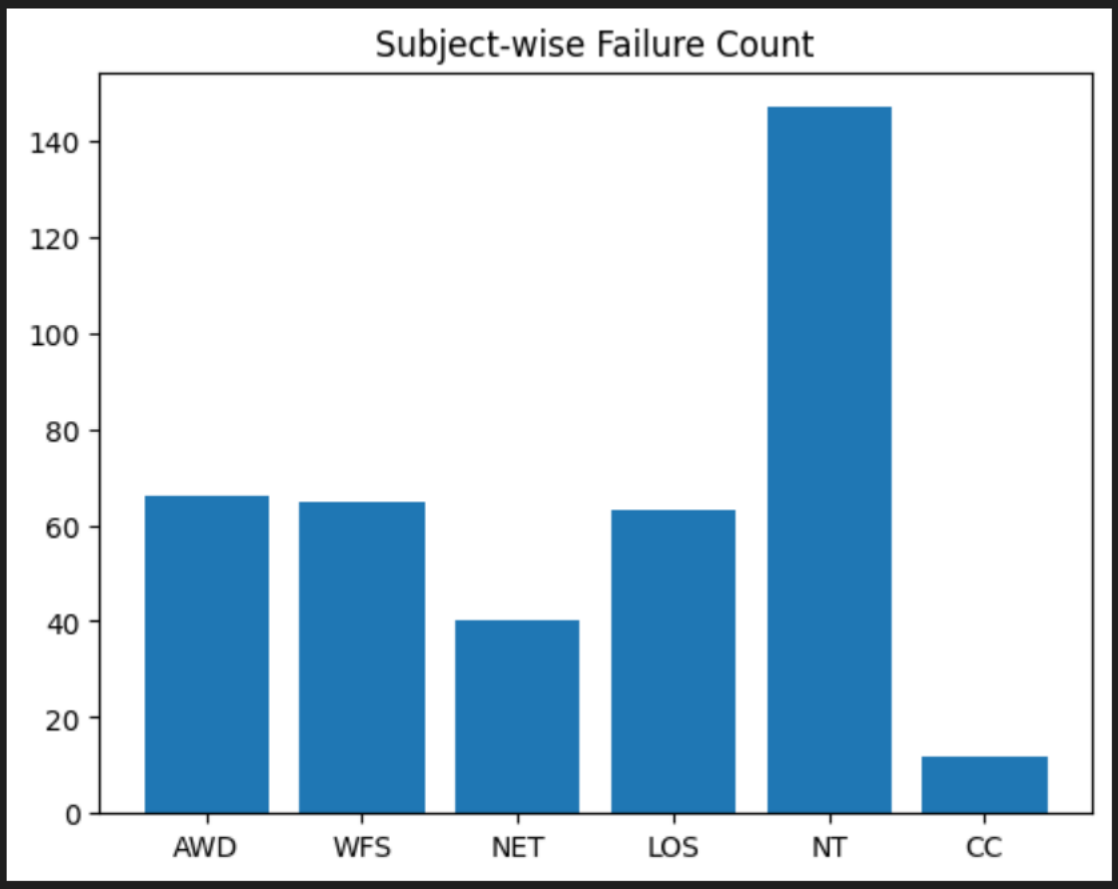
Interpretation:

- Higher PASS count → strong semester performance.
- Low FAIL/ATKT count → difficult subjects.

8.4 Failure Count Per Subject :

```python
import matplotlib.pyplot as plt

# Subject-wise Failure Count based on Passing Criteria
failure_counts = {
    'AWD': (df['AWD_Total'] < 33).sum(),
    'WFS': (df['WFS_Total'] < 33).sum(),
    'NET': (df['.NET_Total'] < 33).sum(),
    'LOS': (df['LOS_Total'] < 33).sum(),
    'NT': (df['NT_Total'] < 33).sum(),
    'CC': (df['CC_Total'] < 18).sum(),
}

# Print Failure Counts
print("Subject-wise Failure Count:")
for subject, count in failure_counts.items():
    print(f"{subject}: {count}")

# Plot Bar Graph
plt.figure(figsize=(8,5))
plt.bar(failure_counts.keys(), failure_counts.values())
plt.title("Subject-wise Failure Count")
plt.xlabel("Subjects")
plt.ylabel("Number of Failed Students")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
{'AWD': 66, 'WFS': 65, 'NET': 40, 'LOS': 63, 'NT': 147, 'CC': 12}
```



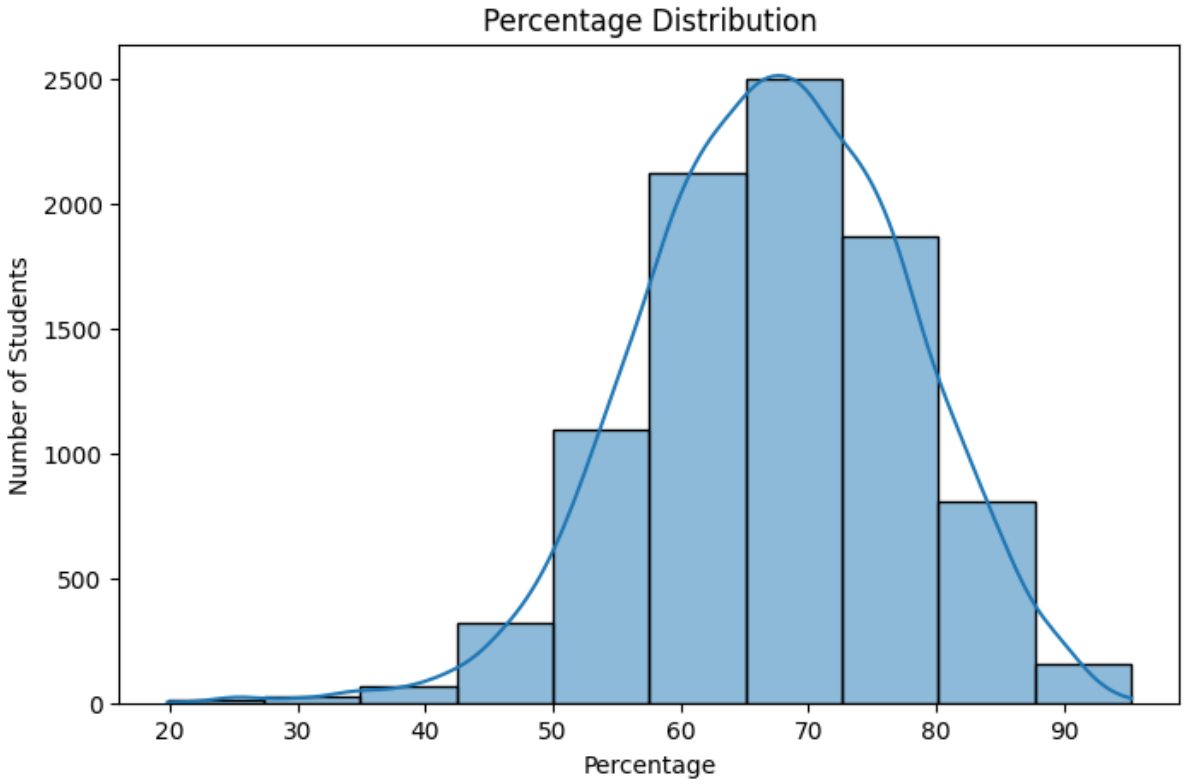
Interpretation:

- Network Technology (NT) has the highest failures (147), indicating it is the most difficult subject.
- AWD, WFS, and LOS show moderate failure counts, suggesting average difficulty.
- Cloud Computing (CC) has the lowest failures (12), showing strongest student performance.

8.6 Percentage Distribution :

Python

```python
plt.figure(figsize=(8,5))
sns.histplot(df['Percentage'], bins=10, kde=True)
plt.title("Percentage Distribution")
plt.xlabel("Percentage")
plt.ylabel("Number of Students")
plt.show()
```



Interpretation :

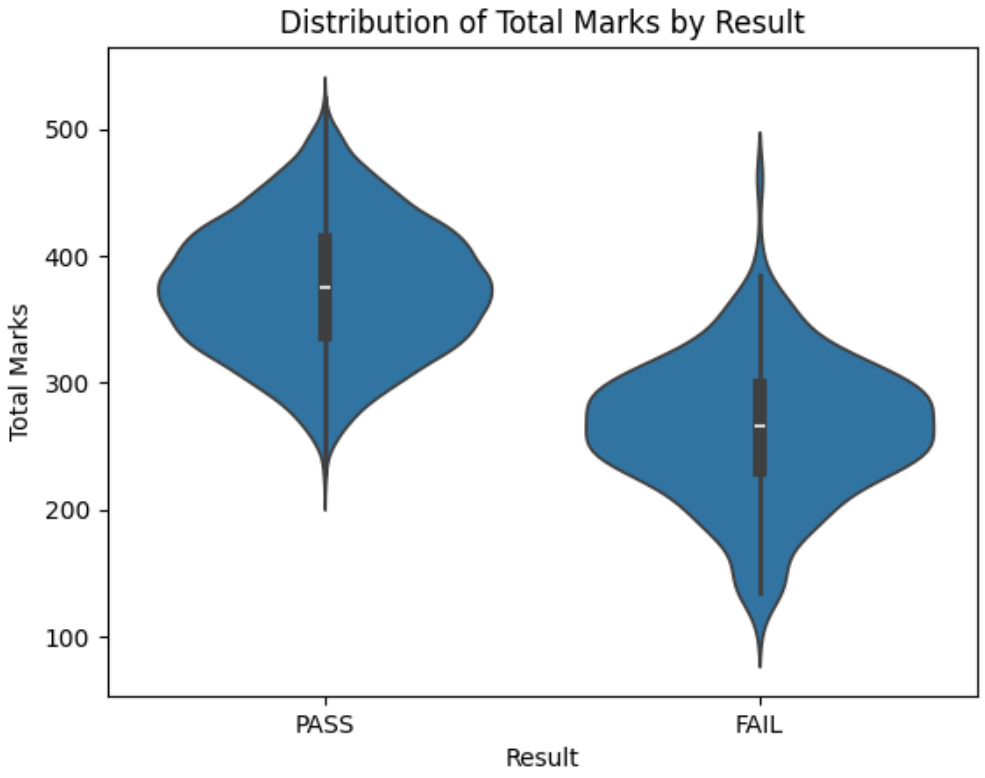
- Most students scored between 55% and 75%, indicating average to good overall performance.
- The distribution appears approximately bell-shaped (normal), showing balanced academic results.
- Very few students scored below 40% or above 90%, indicating limited extreme performance cases.

Chapter 9: Bivariate Analysis

Bivariate Analysis involves examining the relationship between two variables to understand patterns, dependencies, and performance trends within the dataset. In this chapter, we analyze relationships between academic metrics such as total marks, percentage, result status, subject performance, and institutional comparison.

9.1 Distribution of Total Marks by Result :

```python
sns.violinplot(x='Result', y='Total Marks', data=df)
plt.title("Distribution of Total Marks by Result")
plt.show()
```

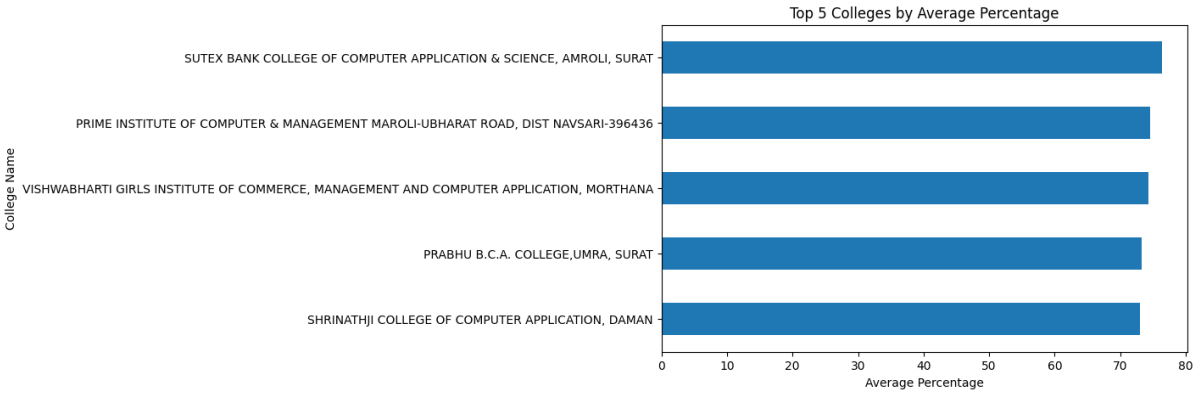


Interpretation:

The violin plot shows a clear distinction between PASS and NOT PASS students. Students who passed are concentrated in the higher total marks range, while non-passing students are clustered in the lower score region. The separation confirms that total marks strongly influence final result status.

9.2 Top 5 Colleges by Average Percentage :

```python
college_avg = df.groupby('College Name')['Percentage'].mean().sort_values(ascending=False)

top5 = college_avg.head(5)

plt.figure(figsize=(8,5))
top5.sort_values().plot(kind='barh')
plt.title("Top 5 Colleges by Average Percentage")
plt.xlabel("Average Percentage")
plt.tight_layout()
plt.show()
```

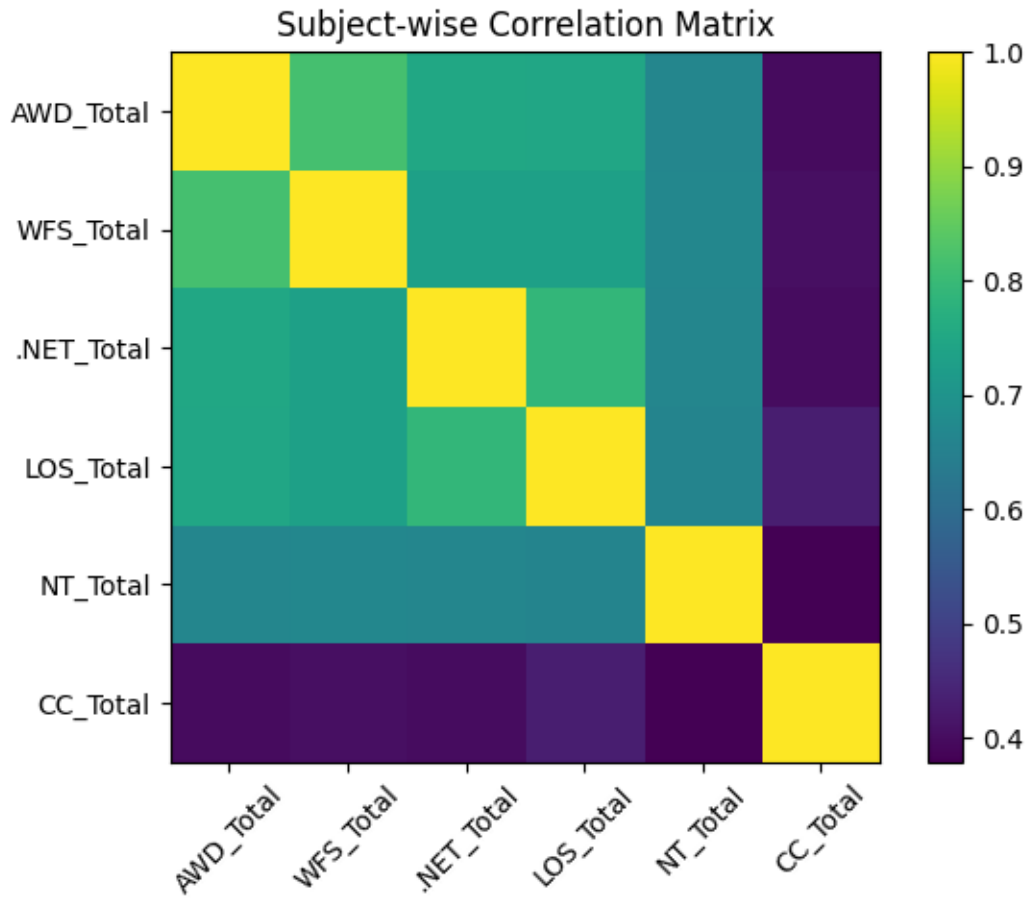


Interpretation:

The comparison reveals measurable differences in academic performance across institutions. The top-performing colleges demonstrate consistently higher average percentages, indicating stronger academic outcomes. This analysis highlights institutional variation under the same university framework.

9.3 Subject-wise Correlation Analysis :

```python
subject_cols = ['AWD_Total','WFS_Total','.NET_Total','LOS_Total','NT_Total','CC_Total']

corr_matrix = df[subject_cols].corr()

plt.figure()
plt.imshow(corr_matrix)
plt.xticks(range(len(subject_cols)), subject_cols, rotation=45)
plt.yticks(range(len(subject_cols)), subject_cols)
plt.title("Subject-wise Correlation Matrix")
plt.colorbar()
plt.show()
```

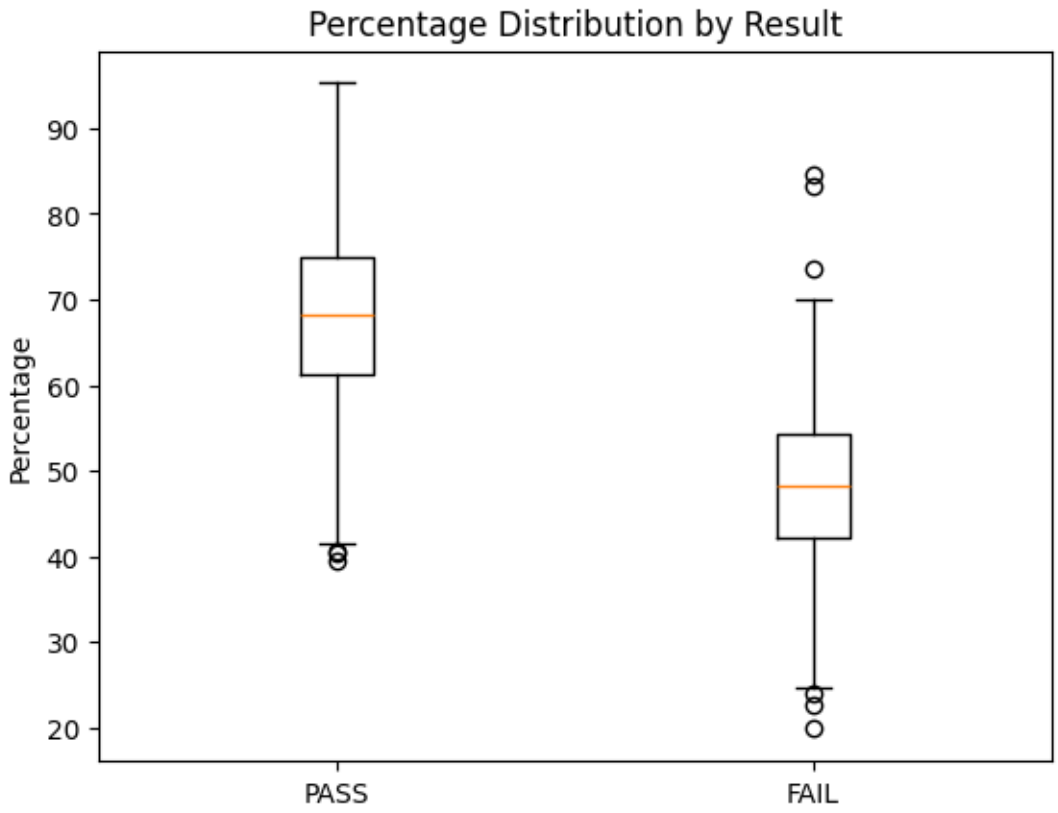


Interpretation:

The correlation heatmap indicates strong positive relationships among several theory-based subjects. High correlation values suggest that students who perform well in one subject tend to perform well in related subjects. Lower correlation values indicate relatively independent performance patterns.

9.4 Percentage Distribution by Result :

```python
pass_percent = df[df['Result'] == 'PASS']['Percentage']
fail_percent = df[df['Result'] != 'PASS']['Percentage']

plt.boxplot([pass_percent, fail_percent], labels=['PASS', 'NOT PASS'])
plt.title("Percentage Distribution by Result")
plt.ylabel("Percentage")
plt.show()
```

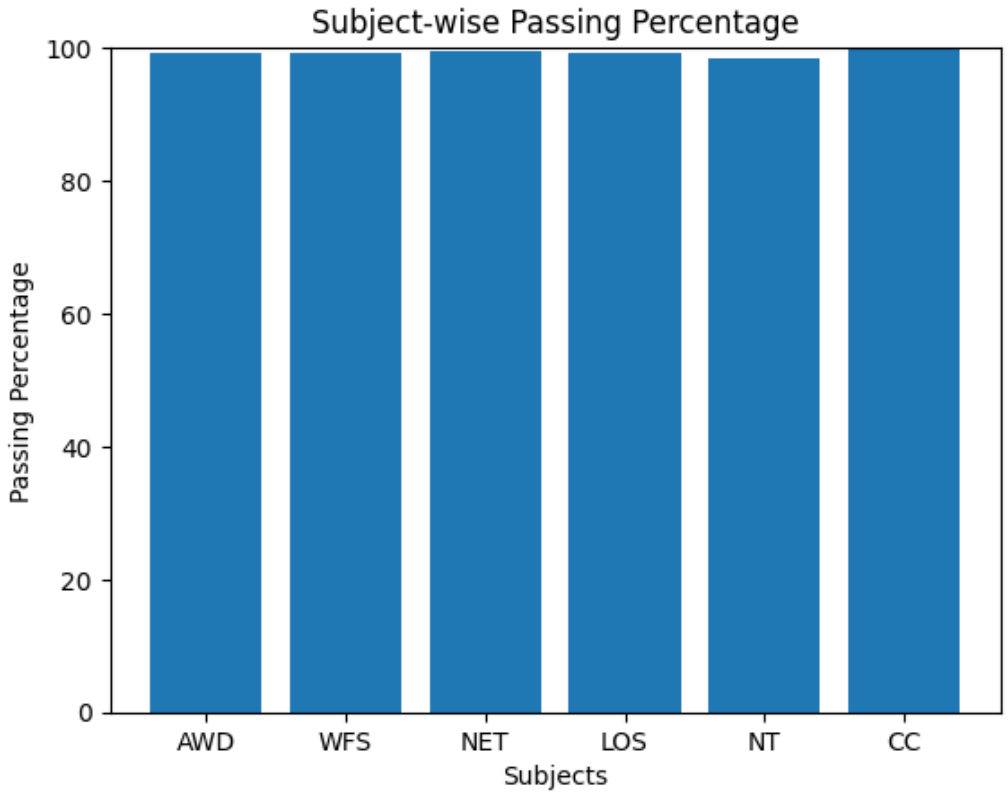


Interpretation:

The median percentage of PASS students is significantly higher than that of NOT PASS students. The boxplot clearly shows that percentage is a strong determinant of final academic outcome. The variability among failing students is greater, indicating inconsistent performance patterns.

9.5 Subject-wise Passing Percentage :

```python
# Total number of students
total_students = len(df)

# Calculate passing percentage per subject
passing_percentage = {
    'AWD': (df['AWD_Total'] >= 33).sum() / total_students * 100,
    'WFS': (df['WFS_Total'] >= 33).sum() / total_students * 100,
    'NET': (df['.NET_Total'] >= 33).sum() / total_students * 100,
    'LOS': (df['LOS_Total'] >= 33).sum() / total_students * 100,
    'NT': (df['NT_Total'] >= 33).sum() / total_students * 100,
    'CC': (df['CC_Total'] >= 18).sum() / total_students * 100   # CC passing criteria
}

# Create Bar Chart
plt.figure()
plt.bar(passing_percentage.keys(), passing_percentage.values())

plt.title("Subject-wise Passing Percentage")
plt.xlabel("Subjects")
plt.ylabel("Passing Percentage")

plt.ylim(0, 100)  # Since percentage scale is 0-100
plt.show()
```



Interpretation:

Most subjects demonstrate high passing percentages, indicating strong overall academic performance. However, comparatively lower passing rates in certain subjects highlight potential areas of academic difficulty. This analysis provides valuable insight for curriculum review and academic improvement strategies.

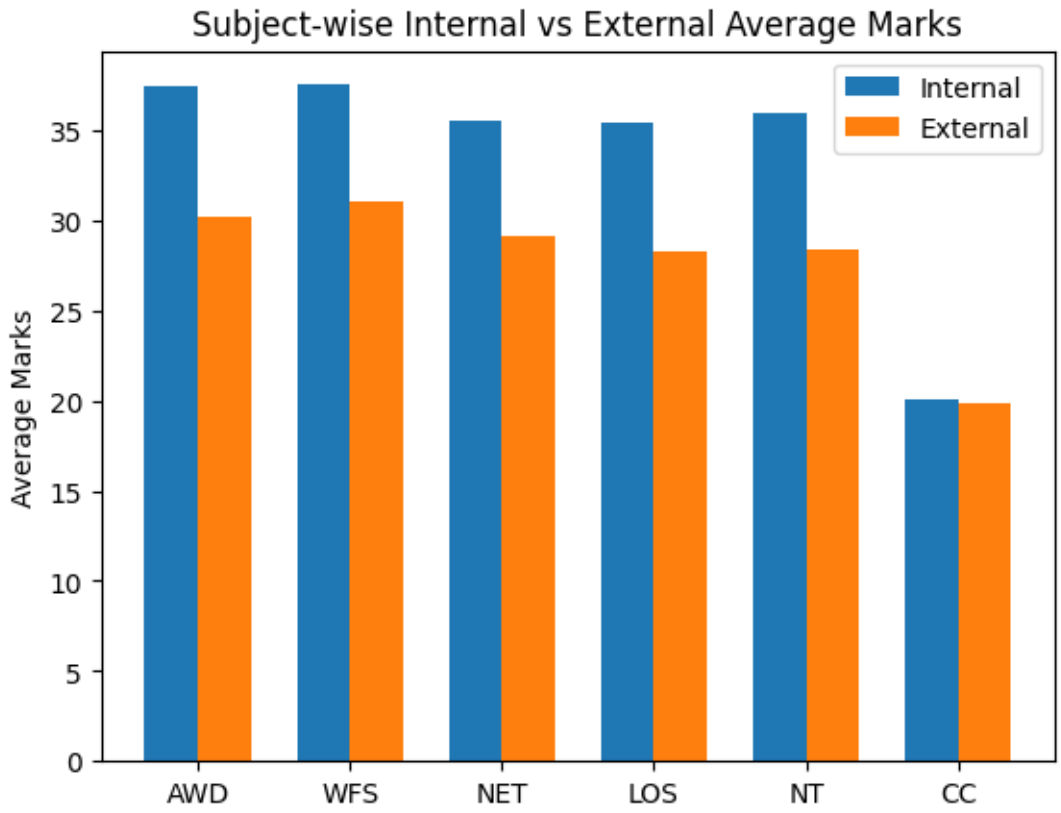
Chapter 10: Multivariate Analysis

Multivariate analysis examines the relationship among three or more variables simultaneously to identify complex patterns within the dataset. In the context of student result analysis, multiple academic indicators such as subject marks, SGPA, percentage, and result status are analyzed together to understand how different factors collectively influence student performance. This approach provides deeper insights compared to univariate or bivariate analysis by considering multiple dimensions of academic data at the same time.

10.1 Subject-wise Internal vs External Average Marks :

```python
subjects = ['AWD','WFS','NET','LOS','NT','CC']

internal_avg = [
df['AWD_Int_Total'].mean(),
df['WFS_Int_Total'].mean(),
df['.NET_Int_Total'].mean(),
df['LOS_Int_Total'].mean(),
df['NT_Int_Total'].mean(),
df['CC_Int_Total'].mean()
]

external_avg = [
df['AWD_Ext_Total'].mean(),
df['WFS_Ext_Total'].mean(),
df['.NET_Ext_Total'].mean(),
df['LOS_Ext_Total'].mean(),
df['NT_Ext_Total'].mean(),
df['CC_Ext_Total'].mean()
]
```

```
x = np.arange(len(subjects))
width = 0.35

plt.figure()

plt.bar(x-width/2, internal_avg, width, label='Internal Marks')
plt.bar(x+width/2, external_avg, width, label='External Marks')

plt.xticks(x, subjects)
plt.xlabel("Subjects")
plt.ylabel("Average Marks")
plt.title("Subject-wise Internal vs External Average Marks")

plt.legend()
plt.show()
```

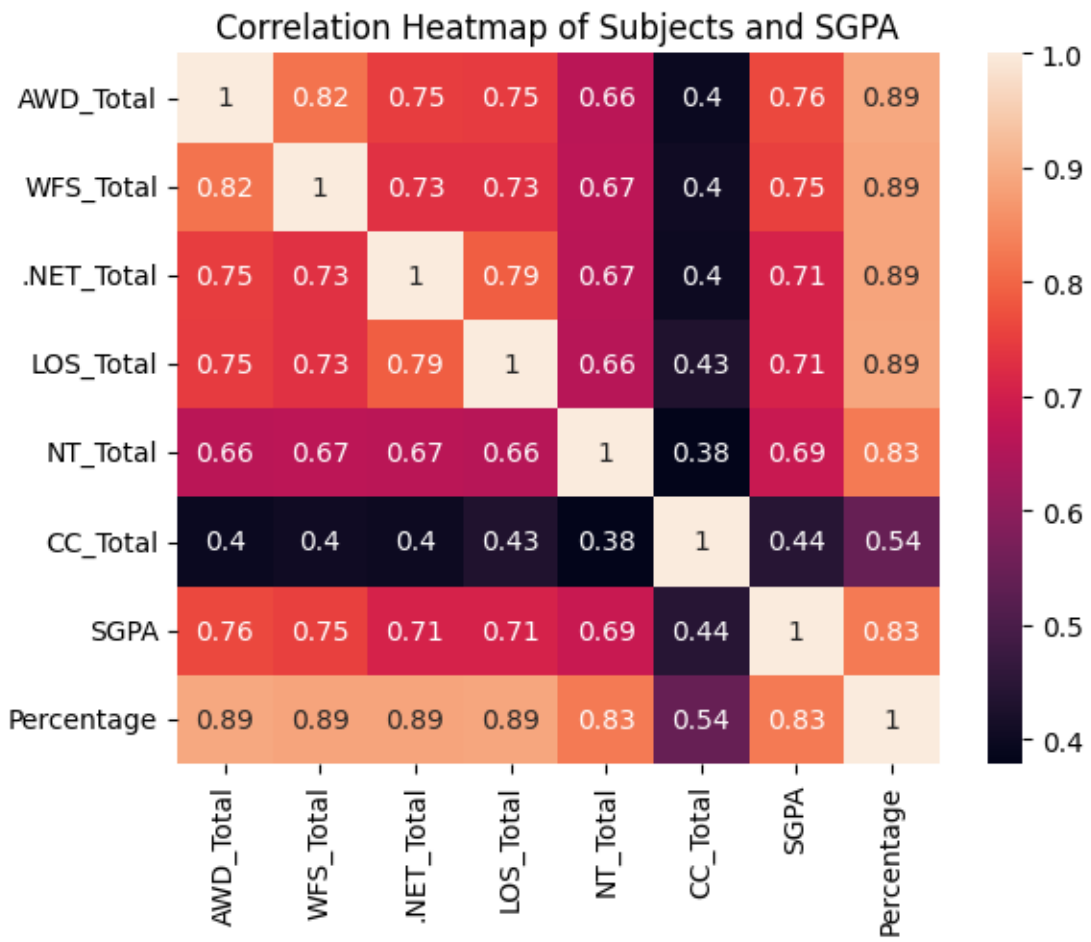


Interpretation :

This visualization compares the average internal and external marks for each subject. The results help identify whether internal assessments are more lenient compared to university examinations. Significant differences between the two can indicate variations in evaluation patterns.

10.2 Correlation Heatmap of Subjects and SGPA :

```python
import seaborn as sns
import matplotlib.pyplot as plt

columns = [
'AWD_Total','WFS_Total','.NET_Total',
'LOS_Total','NT_Total','CC_Total',
'SGPA','Percentage'
]

plt.figure()

sns.heatmap(df[columns].corr(), annot=True)

plt.title("Correlation Heatmap of Subjects and SGPA")

plt.show()
```



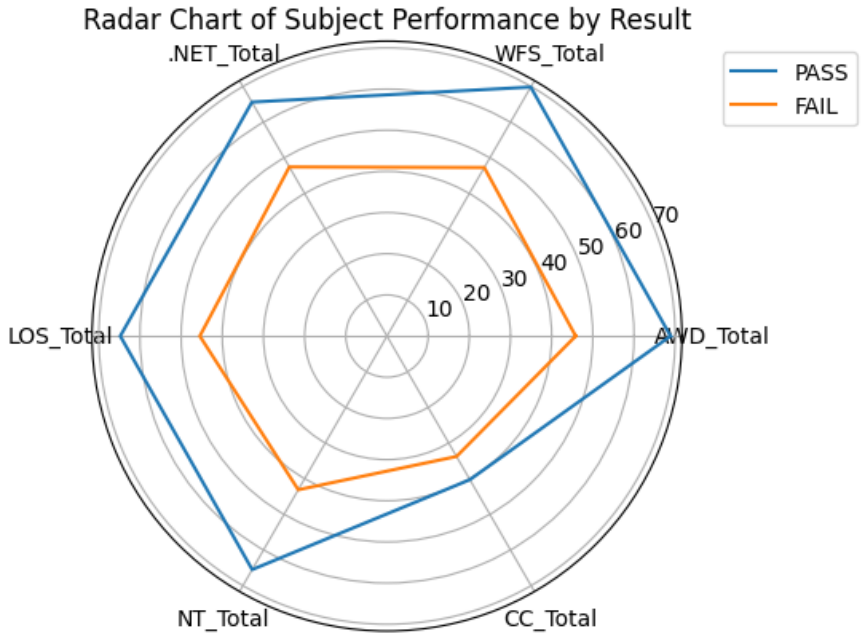
Interpretation :

The heatmap illustrates the strength of relationships between subject marks, SGPA, and percentage. Higher positive correlations suggest that strong performance in those subjects contributes significantly to overall academic achievement.

10.3 Radar Chart of Subject Performance by Result :

```python
import numpy as np
import matplotlib.pyplot as plt

subjects = ['AWD_Total','WFS_Total','.NET_Total','LOS_Total','NT_Total','CC_Total']

pass_avg = df[df['Result']=='PASS'][subjects].mean().values
fail_avg = df[df['Result']!='PASS'][subjects].mean().values

angles = np.linspace(0,2*np.pi,len(subjects),endpoint=False)

pass_avg = np.concatenate((pass_avg,[pass_avg[0]]))
fail_avg = np.concatenate((fail_avg,[fail_avg[0]]))
angles = np.concatenate((angles,[angles[0]]))

fig = plt.figure()
ax = fig.add_subplot(111, polar=True)

ax.plot(angles, pass_avg, label='PASS')
ax.plot(angles, fail_avg, label='FAIL')
```

```python
ax.set_xticks(angles[:-1])
ax.set_xticklabels(subjects)

plt.title("Radar Chart of Subject Performance by Result")

# Move legend outside
plt.legend(loc='upper left', bbox_to_anchor=(1.05,1))

plt.show()
```

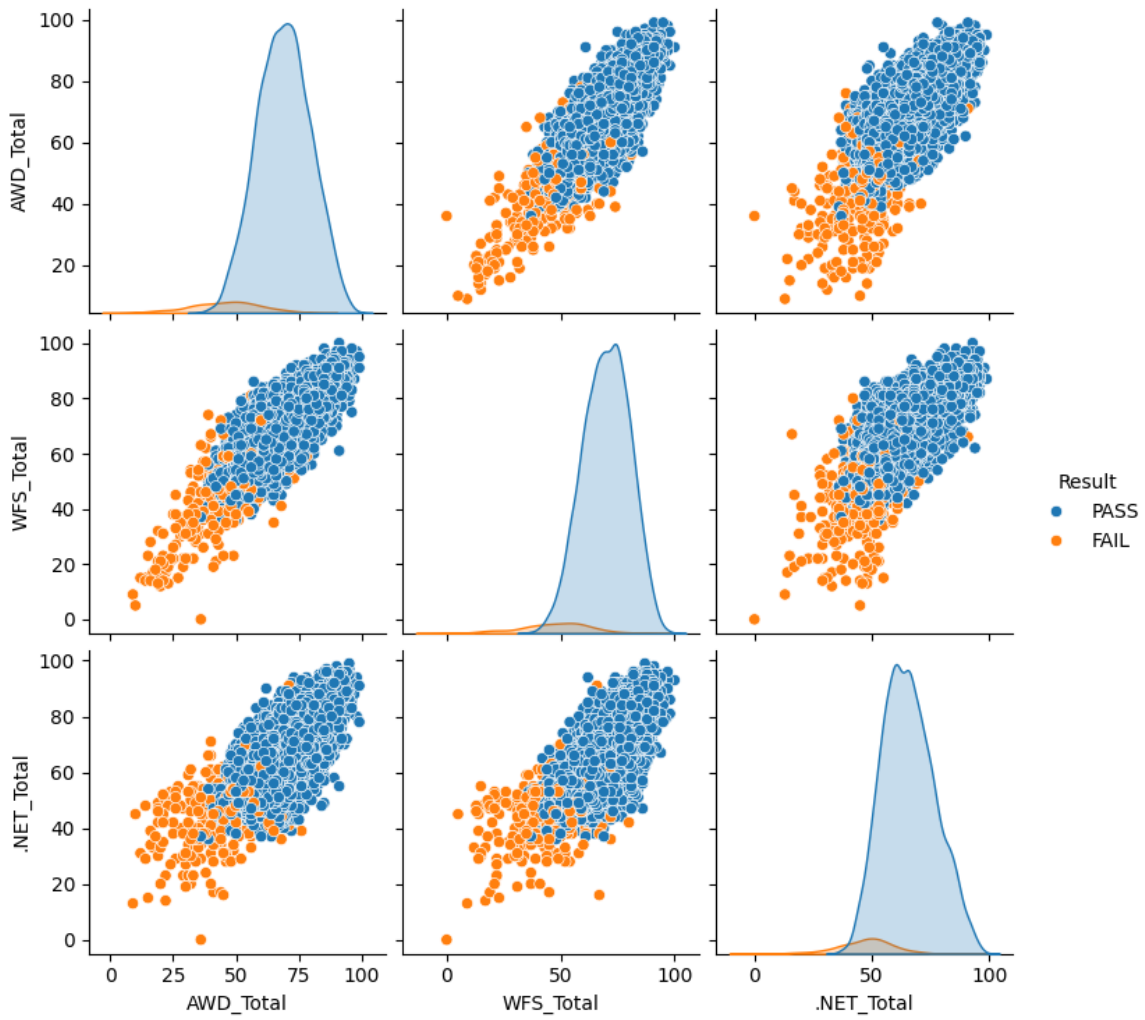


Interpretation :

The radar chart visualizes differences in subject-wise performance between passing and failing students. Subjects where failing students score significantly lower indicate areas that may contribute most to academic failure.

10.4 Pair Plot (Scatter Matrix) of Core Subjects :

```python
import seaborn as sns

sns.pairplot(
df[['AWD_Total','WFS_Total','.NET_Total','Result']],
hue='Result'
)

plt.show()
```



Interpretation

The pair plot provides a comprehensive view of relationships between core subjects. The color differentiation highlights clusters of passing and failing students, helping identify subject performance patterns associated with academic success.

Chapter 11: Data Visualization Using Sweetviz

Data visualization plays a crucial role in understanding patterns, trends, and relationships within a dataset. Instead of manually creating multiple charts, the Sweetviz library was used to generate an automated exploratory data analysis (EDA) report. Sweetviz provides an interactive visualization dashboard that summarizes dataset characteristics including distributions, correlations, missing values, and variable relationships.

The generated report provides a comprehensive overview of student performance data in an easy-to-understand visual format.

11.1 Sweetviz Library Overview :

Sweetviz is a Python library designed for automated exploratory data analysis. It generates an interactive HTML report containing statistical summaries and visualizations for all variables in a dataset.

Key features include:

- Variable distribution visualization
- Target variable analysis
- Correlation analysis
- Missing value detection
- Feature comparison
- Automatic chart generation

This tool significantly reduces the effort required to manually create multiple charts for dataset exploration.

```python
import sweetviz as sv

report = sv.analyze(df)

report.show_html("student_result_analysis.html")
```

SweetViz
2.3.1
Get updates, docs & report issues here
Created & maintained by Francois Bertrand
Graphic design by Jean-Francois Hains

DataFrame

NO COMPARISON TARGET

ASSOCIATIONS

8987	ROWS
10	DUPLICATES
6.1 MB	RAM
42	FEATURES
3	CATEGORICAL
38	NUMERICAL
1	TEXT

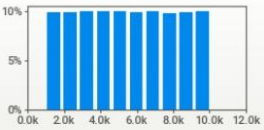
DataFrame

1 Seat Number

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	8,977 (>99%)
ZEROES:	---

MAX	10,042
95%	9,592
Q3	7,772
AVG	5,520
MEDIAN	5,513
Q1	3,264
5%	1,455
MIN	1,001

RANGE	9,041
IQR	4,507
STD	2,609
VAR	6.8M
KURT.	-1.20
SKEW	0.003
SUM	49.6M



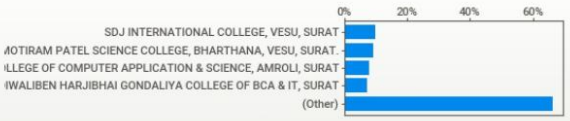
2 Name

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	8,965 (>99%)

2	<1%	PATEL PRINCE NILESHBHAI
2	<1%	JOGANI MITESH BHUPATBHAI
2	<1%	RADADIYA PRINCE ASHOKBHAI
2	<1%	KAKADIYA AASTHA BHAVESHBHAI
2	<1%	KADIYA MANSIBEN KETANBHAI
2	<1%	DHAMELIYA PRINCEKUMAR BHARATBHAI
2	<1%	KACHHADIYA PARTH DINESHBHAI
8,973	>99%	(Other)

3 College Name

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	36 (<1%)

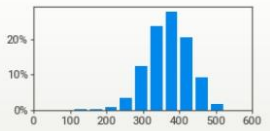


4 Total Marks

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	349 (4%)
ZEROES:	---

MAX	524
95%	461
Q3	410
MEDIAN	371
AVG	370
Q1	332
5%	277
MIN	109

RANGE	415
IQR	78.0
STD	57.2
VAR	3,271
KURT.	0.278
SKEW	-0.285
SUM	3.3M

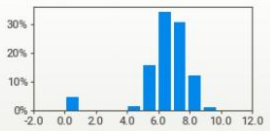


5 SGPA

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	64 (<1%)
ZEROES:	408 (5%)

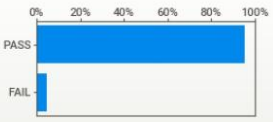
MAX	9.82
95%	8.36
Q3	7.45
MEDIAN	6.73
AVG	6.47
Q1	6.00
5%	4.55
MIN	0.00

RANGE	9.82
IQR	1.45
STD	1.69
VAR	2.86
KURT.	7.07
SKEW	-2.37
SUM	58,133



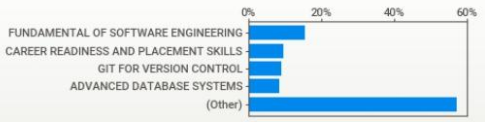
6 Result

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	2 (<1%)



7 Course_Name

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	23 (<1%)

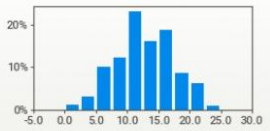


8 AWD_Ext_Th

VALUES:	8,987 (100%)
MISSING:	---
DISTINCT:	26 (<1%)
ZEROES:	37 (<1%)

MAX	25.0
95%	20.0
Q3	16.0
MEDIAN	13.0
AVG	12.6
Q1	9.0
5%	5.0
MIN	0.0

RANGE	25.0
IQR	7.00
STD	4.65
VAR	21.6
KURT.	-0.419
SKEW	-0.016
SUM	113k

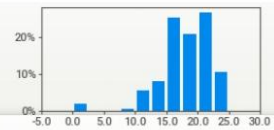


9 AWD_Ext_Pr

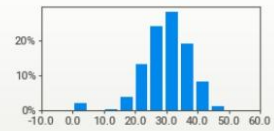
VALUES:	8,987 (100%)
MISSING:	---

MAX	25.0
95%	23.0

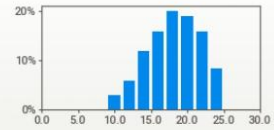
RANGE	25.0
IQR	4.00

DISTINCT:	21 (<1%)	Q3	20.0	STD	4.24	
ZEROES:	182 (2%)	MEDIAN	18.0	VAR	18.0	
		AVG	17.8			
		Q1	16.0	KURT.	4.29	
		5%	11.0	SKEW	-1.47	
		MIN	0.0	SUM	160k	

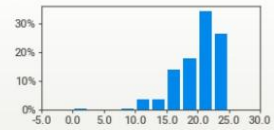
10 AWD_Ext_Total

VALUES:	8,987 (100%)	MAX	49.0	RANGE	49.0	
MISSING:	---	95%	42.0	IQR	9.00	
DISTINCT:	41 (<1%)	Q3	35.0	STD	7.85	
		MEDIAN	31.0	VAR	61.6	
ZEROES:	194 (2%)	AVG	30.1			
		Q1	26.0	KURT.	2.94	
		5%	19.0	SKEW	-1.07	
		MIN	0.0	SUM	271k	

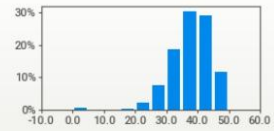
11 AWD_Int_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	20.0	
MISSING:	---	95%	23.0	IQR	5.00	
DISTINCT:	21 (<1%)	Q3	20.0	STD	3.60	
		MEDIAN	18.0	VAR	13.0	
ZEROES:	---	AVG	17.6			
		Q1	15.0	KURT.	-0.561	
		5%	11.0	SKEW	-0.238	
		MIN	5.0	SUM	158k	

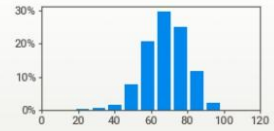
12 AWD_Int_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0	
MISSING:	---	95%	24.0	IQR	5.00	
DISTINCT:	19 (<1%)	Q3	23.0	STD	3.74	
		MEDIAN	20.0	VAR	14.0	
ZEROES:	45 (<1%)	AVG	19.8			
		Q1	18.0	KURT.	3.31	
		5%	13.0	SKEW	-1.32	
		MIN	0.0	SUM	178k	

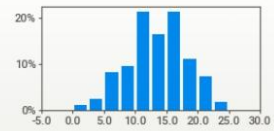
13 AWD_Int_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0	
MISSING:	---	95%	47.0	IQR	8.00	
DISTINCT:	36 (<1%)	Q3	42.0	STD	6.51	
		MEDIAN	38.0	VAR	42.4	
ZEROES:	45 (<1%)	AVG	37.4			
		Q1	34.0	KURT.	4.45	
		5%	27.0	SKEW	-1.20	
		MIN	0.0	SUM	336k	

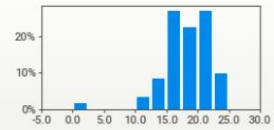
14 AWD_Total

VALUES:	8,987 (100%)	MAX	99.0	RANGE	90.0	
MISSING:	---	95%	86.0	IQR	16.0	
DISTINCT:	89 (<1%)	Q3	76.0	STD	11.7	
		MEDIAN	68.0	VAR	138	
ZEROES:	---	AVG	67.8			
		Q1	60.0	KURT.	0.802	
		5%	49.0	SKEW	-0.451	
		MIN	9.0	SUM	610k	

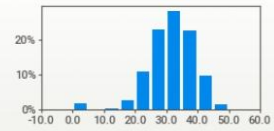
15 WFS_Ext_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0	
MISSING:	---	95%	21.0	IQR	7.00	
DISTINCT:	26 (<1%)	Q3	17.0	STD	4.69	
		AVG	13.3	VAR	22.0	
ZEROES:	39 (<1%)	MEDIAN	13.0			
		Q1	10.0	KURT.	-0.334	
		5%	5.0	SKEW	-0.169	
		MIN	0.0	SUM	119k	

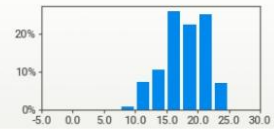
16 WFS_Ext_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0	
MISSING:	---	95%	23.0	IQR	4.00	
DISTINCT:	19 (<1%)	Q3	20.0	STD	3.95	
		MEDIAN	18.0	VAR	15.6	
ZEROES:	157 (2%)	AVG	17.9			
		Q1	16.0	KURT.	5.54	
		5%	12.0	SKEW	-1.56	
		MIN	0.0	SUM	161k	

17 WFS_Ext_Total

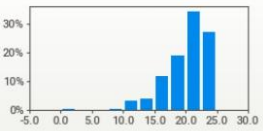
VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0	
MISSING:	---	95%	42.0	IQR	9.00	
DISTINCT:	41 (<1%)	Q3	36.0	STD	7.60	
		MEDIAN	32.0	VAR	57.8	
ZEROES:	168 (2%)	AVG	31.1			
		Q1	27.0	KURT.	3.47	
		5%	20.0	SKEW	-1.17	
		MIN	0.0	SUM	279k	

18 WFS_Int_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0	
MISSING:	---	95%	23.0	IQR	5.00	
DISTINCT:	24 (<1%)	Q3	20.0	STD	3.45	
		MEDIAN	18.0	VAR	11.9	
ZEROES:	1 (<1%)	AVG	17.7			
		Q1	15.0	KURT.	-0.196	
		5%	12.0	SKEW	-0.332	
		MIN	0.0	SUM	159k	

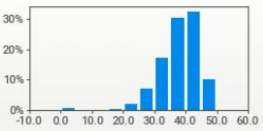
19 WFS_Int_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	24.0	IQR	5.00		
		Q3	23.0	STD	3.69		
DISTINCT:	20 (<1%)	MEDIAN	21.0	VAR	13.6		
		AVG	19.9				
ZEROES:	47 (<1%)	Q1	18.0	KURT.	3.78		
		5%	13.0	SKEW	-1.42		
		MIN	0.0	SUM	179k		



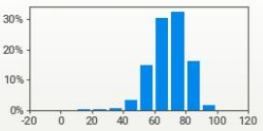
20 WFS_Int_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0		
MISSING:	---	95%	46.0	IQR	8.00		
		Q3	42.0	STD	6.32		
DISTINCT:	36 (<1%)	MEDIAN	38.0	VAR	39.9		
		AVG	37.5				
ZEROES:	47 (<1%)	Q1	34.0	KURT.	5.51		
		5%	27.0	SKEW	-1.44		
		MIN	0.0	SUM	337k		



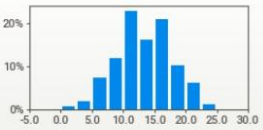
21 WFS_Total

VALUES:	8,987 (100%)	MAX	100	RANGE	100		
MISSING:	---	95%	86	IQR	15.0		
		Q3	77	STD	11.5		
DISTINCT:	88 (<1%)	MEDIAN	70	VAR	133		
		AVG	69				
ZEROES:	1 (<1%)	Q1	62	KURT.	1.39		
		5%	50	SKEW	-0.666		
		MIN	0	SUM	618k		



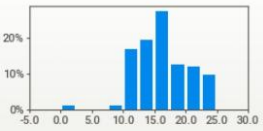
22 .NET_Ext_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	20.0	IQR	6.00		
		Q3	16.0	STD	4.41		
DISTINCT:	26 (<1%)	AVG	13.1	VAR	19.5		
		MEDIAN	13.0				
ZEROES:	39 (<1%)	Q1	10.0	KURT.	-0.278		
		5%	6.0	SKEW	-0.087		
		MIN	0.0	SUM	118k		



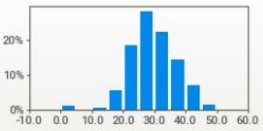
23 .NET_Ext_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	24.0	IQR	6.00		
		Q3	19.0	STD	4.31		
DISTINCT:	21 (<1%)	AVG	16.0	VAR	18.6		
		MEDIAN	15.0				
ZEROES:	103 (1%)	Q1	13.0	KURT.	1.02		
		5%	11.0	SKEW	-0.112		
		MIN	0.0	SUM	144k		



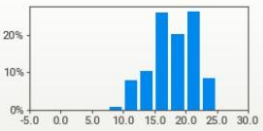
24 .NET_Ext_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0		
MISSING:	---	95%	42.0	IQR	10.0		
		Q3	34.0	STD	7.60		
DISTINCT:	41 (<1%)	AVG	29.1	VAR	57.7		
		MEDIAN	29.0				
ZEROES:	113 (1%)	Q1	24.0	KURT.	1.51		
		5%	18.0	SKEW	-0.392		
		MIN	0.0	SUM	261k		



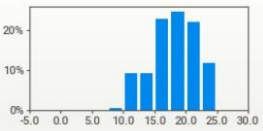
25 .NET_Int_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	23.0	IQR	5.00		
		Q3	20.0	STD	3.60		
DISTINCT:	23 (<1%)	MEDIAN	18.0	VAR	12.9		
		AVG	17.7				
ZEROES:	1 (<1%)	Q1	15.0	KURT.	-0.357		
		5%	11.0	SKEW	-0.297		
		MIN	0.0	SUM	159k		



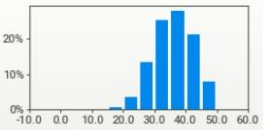
26 .NET_Int_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	24.0	IQR	6.00		
		Q3	21.0	STD	3.83		
DISTINCT:	22 (<1%)	MEDIAN	18.0	VAR	14.7		
		AVG	17.9				
ZEROES:	11 (<1%)	Q1	15.0	KURT.	-0.064		
		5%	11.0	SKEW	-0.329		
		MIN	0.0	SUM	161k		



27 .NET_Int_Total

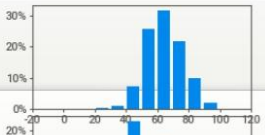
VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0		
MISSING:	---	95%	46.0	IQR	9.00		
		Q3	40.0	STD	6.41		
DISTINCT:	38 (<1%)	MEDIAN	36.0	VAR	41.1		
		AVG	35.6				
ZEROES:	11 (<1%)	Q1	31.0	KURT.	0.601		
		5%	25.0	SKEW	-0.354		
		MIN	0.0	SUM	320k		



28 .NET_Total

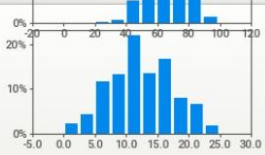
VALUES:	8,987 (100%)	MAX	99.0	RANGE	99.0		
MISSING:	---	95%	85.0	IQR	17.0		
		Q3	73.0	STD	11.9		

DISTINCT:	83 (<1%)	AVG	64.8	VAR	141
ZEROES:	1 (<1%)	MEDIAN	64.0	KURT.	0.092
		Q1	56.0	SKEW	0.034
		5%	47.0	SUM	582k
		MIN	0.0		



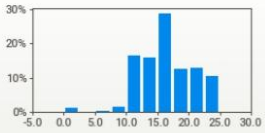
29 LOS_Ext_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	21.0	IQR	7.00
		Q3	16.0	STD	5.05
DISTINCT:	26 (<1%)	AVG	12.2	VAR	25.5
		MEDIAN	12.0		
ZEROES:	43 (<1%)	Q1	9.0	KURT.	-0.524
		5%	4.0	SKEW	0.041
		MIN	0.0	SUM	110k



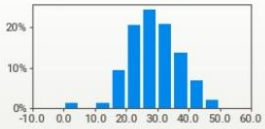
30 LOS_Ext_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	24.0	IQR	6.00
		Q3	19.0	STD	4.42
DISTINCT:	25 (<1%)	AVG	16.2	VAR	19.6
		MEDIAN	16.0		
ZEROES:	105 (1%)	Q1	13.0	KURT.	0.829
		5%	10.0	SKEW	-0.214
		MIN	0.0	SUM	145k



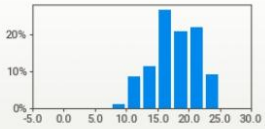
31 LOS_Ext_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	42.0	IQR	11.0
		Q3	34.0	STD	8.10
DISTINCT:	43 (<1%)	AVG	28.3	VAR	65.6
		MEDIAN	28.0		
ZEROES:	117 (1%)	Q1	23.0	KURT.	0.767
		5%	16.0	SKEW	-0.273
		MIN	0.0	SUM	255k



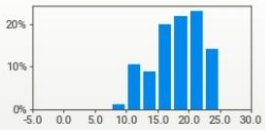
32 LOS_Int_Th

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	23.7	IQR	5.00
		Q3	20.0	STD	3.66
DISTINCT:	22 (<1%)	MEDIAN	18.0	VAR	13.4
		AVG	17.5		
ZEROES:	1 (<1%)	Q1	15.0	KURT.	-0.479
		5%	11.0	SKEW	-0.184
		MIN	0.0	SUM	157k



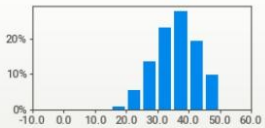
33 LOS_Int_Pr

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0
MISSING:	---	95%	25.0	IQR	6.00
		Q3	21.0	STD	4.10
DISTINCT:	21 (<1%)	MEDIAN	18.0	VAR	16.8
		AVG	18.0		
ZEROES:	8 (<1%)	Q1	15.0	KURT.	-0.395
		5%	11.0	SKEW	-0.331
		MIN	0.0	SUM	161k



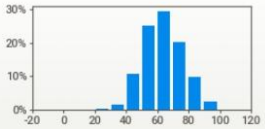
34 LOS_Int_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	46.0	IQR	9.00
		Q3	40.0	STD	6.92
DISTINCT:	39 (<1%)	MEDIAN	36.0	VAR	47.9
		AVG	35.5		
ZEROES:	9 (<1%)	Q1	31.0	KURT.	0.104
		5%	23.0	SKEW	-0.303
		MIN	0.0	SUM	319k



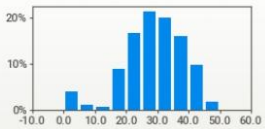
35 LOS_Total

VALUES:	8,987 (100%)	MAX	99.0	RANGE	99.0
MISSING:	---	95%	86.0	IQR	18.0
		Q3	73.0	STD	12.9
DISTINCT:	85 (<1%)	MEDIAN	64.0	VAR	165
		AVG	63.9		
ZEROES:	9 (<1%)	Q1	55.0	KURT.	0.341
		5%	44.0	SKEW	-0.101
		MIN	0.0	SUM	574k



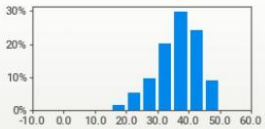
36 NT_Ext_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	42.0	IQR	12.0
		Q3	35.0	STD	9.66
DISTINCT:	48 (<1%)	MEDIAN	29.0	VAR	93.3
		AVG	28.4		
ZEROES:	334 (4%)	Q1	23.0	KURT.	1.09
		5%	9.0	SKEW	-0.800
		MIN	0.0	SUM	255k



37 NT_Int_Total

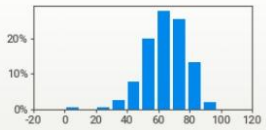
VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0
MISSING:	---	95%	46.0	IQR	9.00
		Q3	41.0	STD	6.80
DISTINCT:	37 (<1%)	MEDIAN	37.0	VAR	46.2
		AVG	35.9		
ZEROES:	1 (<1%)	Q1	32.0	KURT.	-0.032
		5%	22.0	SKEW	-0.543
		MIN	0.0	SUM	323k



NT_Total

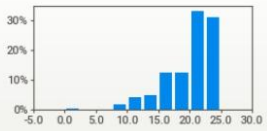
38

VALUES:	8,987 (100%)	MAX	98.0	RANGE	98.0		
MISSING:	---	95%	85.0	IQR	18.0		
		Q3	74.0	STD	13.8		
DISTINCT:	81 (<1%)	MEDIAN	65.0	VAR	192		
		AVG	64.5				
ZEROES:	49 (<1%)	Q1	56.0	KURT.	1.78		
		5%	42.0	SKEW	-0.730		
		MIN	0.0	SUM	580k		



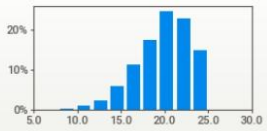
39 CC_Ext_Total

VALUES:	8,987 (100%)	MAX	25.0	RANGE	25.0		
MISSING:	---	95%	25.0	IQR	5.00		
		Q3	23.0	STD	4.11		
DISTINCT:	23 (<1%)	MEDIAN	21.0	VAR	16.9		
		AVG	19.9				
ZEROES:	31 (<1%)	Q1	18.0	KURT.	1.75		
		5%	12.0	SKEW	-1.21		
		MIN	0.0	SUM	179k		



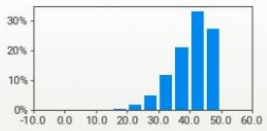
40 CC_Int_Total

VALUES:	8,987 (100%)	MAX	25.0	RANGE	19.0		
MISSING:	---	95%	25.0	IQR	5.00		
		Q3	23.0	STD	3.25		
DISTINCT:	19 (<1%)	AVG	20.1	VAR	10.6		
		MEDIAN	20.0				
ZEROES:	---	Q1	18.0	KURT.	0.188		
		5%	14.0	SKEW	-0.682		
		MIN	6.0	SUM	180k		



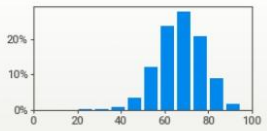
41 CC_Total

VALUES:	8,987 (100%)	MAX	50.0	RANGE	50.0		
MISSING:	---	95%	48.0	IQR	9.00		
		Q3	45.0	STD	6.37		
DISTINCT:	38 (<1%)	MEDIAN	41.0	VAR	40.6		
		AVG	40.0				
ZEROES:	1 (<1%)	Q1	36.0	KURT.	0.663		
		5%	28.0	SKEW	-0.890		
		MIN	0.0	SUM	359k		



42 Percentage

VALUES:	8,987 (100%)	MAX	95.3	RANGE	75.5		
MISSING:	---	95%	83.8	IQR	14.2		
		Q3	74.5	STD	10.4		
DISTINCT:	349 (4%)	MEDIAN	67.5	VAR	108		
		AVG	67.3				
ZEROES:	---	Q1	60.4	KURT.	0.278		
		5%	50.4	SKEW	-0.285		
		MIN	19.8	SUM	605k		



Chapter 12 Machine Learning Implementation :

In this chapter, the Python implementation used for student performance clustering is presented.
The code performs the following steps:

- Import required libraries
- Load cleaned dataset
- Check missing values
- Standardize features
- Apply K-Means Clustering
- Determine optimal clusters using Elbow Method
- Assign clusters to students
- Label clusters based on performance
- Visualize clusters using PCA
- Evaluate clustering using Silhouette Score
- Save trained model and results

12.1 Import Required Libraries :

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import joblib
```

12.2 Load Dataset :

```python
# Step 2: Load cleaned dataset

file_path = "D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\Data\\processed\\clea

df = pd.read_excel(file_path)

print(df.head())
print("Dataset Shape:", df.shape)
```

12.3 Select Features for Clustering :

```python
feature_cols = [
    "AWD_Total",
    "WFS_Total",
    ".NET_Total",
    "LOS_Total",
    "NT_Total",
    "CC_Total"
]

X = df[feature_cols]
```

12.4 Check Missing Values :

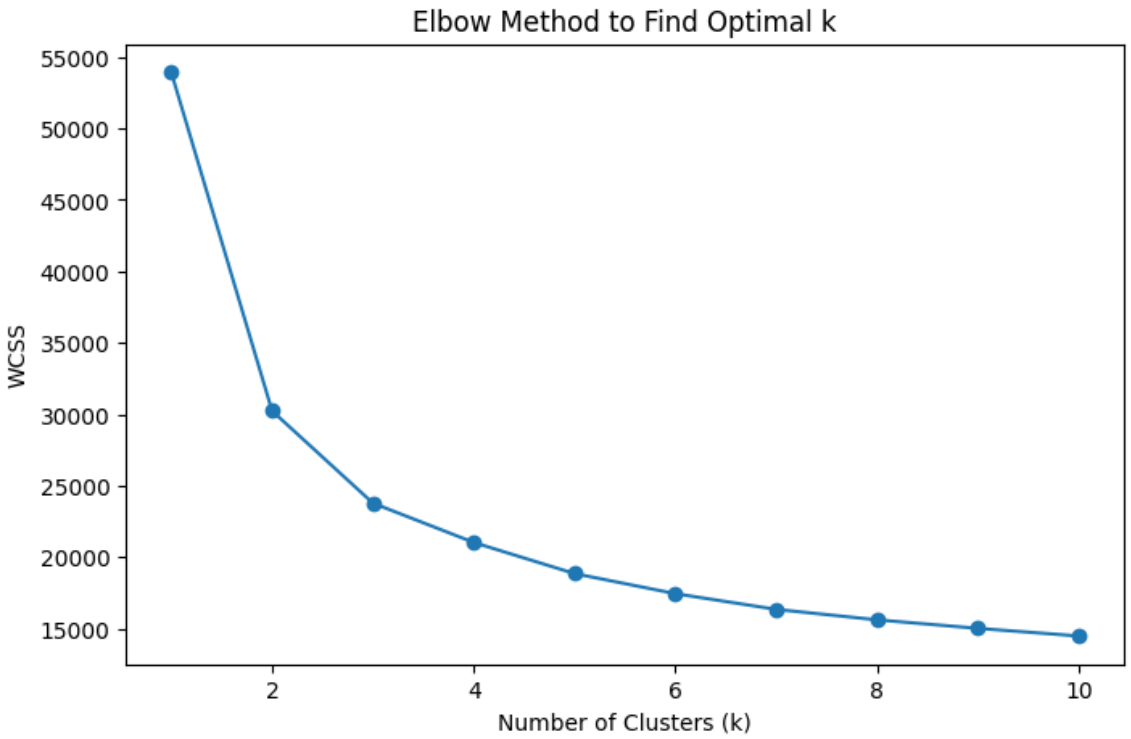
```python
print("Missing values in each column:")
print(X.isnull().sum())
```

12.5 Standardize the Data :

```python
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

print("Mean of scaled features:")
print(X_scaled_df.mean())

print("Standard deviation of scaled features:")
print(X_scaled_df.std())
```

12.6 Determine Optimal Clusters using Elbow Method :

```python
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method for Optimal Clusters")
plt.show()
```



12.7 Apply K-Means Clustering :

```python
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

clusters = kmeans.fit_predict(X_scaled)

df["Cluster"] = clusters
```

12.8 Count Students in Each Cluster :

```python
print(df["Cluster"].value_counts())
```

12.9 Cluster-wise Performance Summary :

```python
cluster_summary = df.groupby("Cluster")[feature_cols].mean()

print(cluster_summary)
```

12.10 Assign Performance Labels :

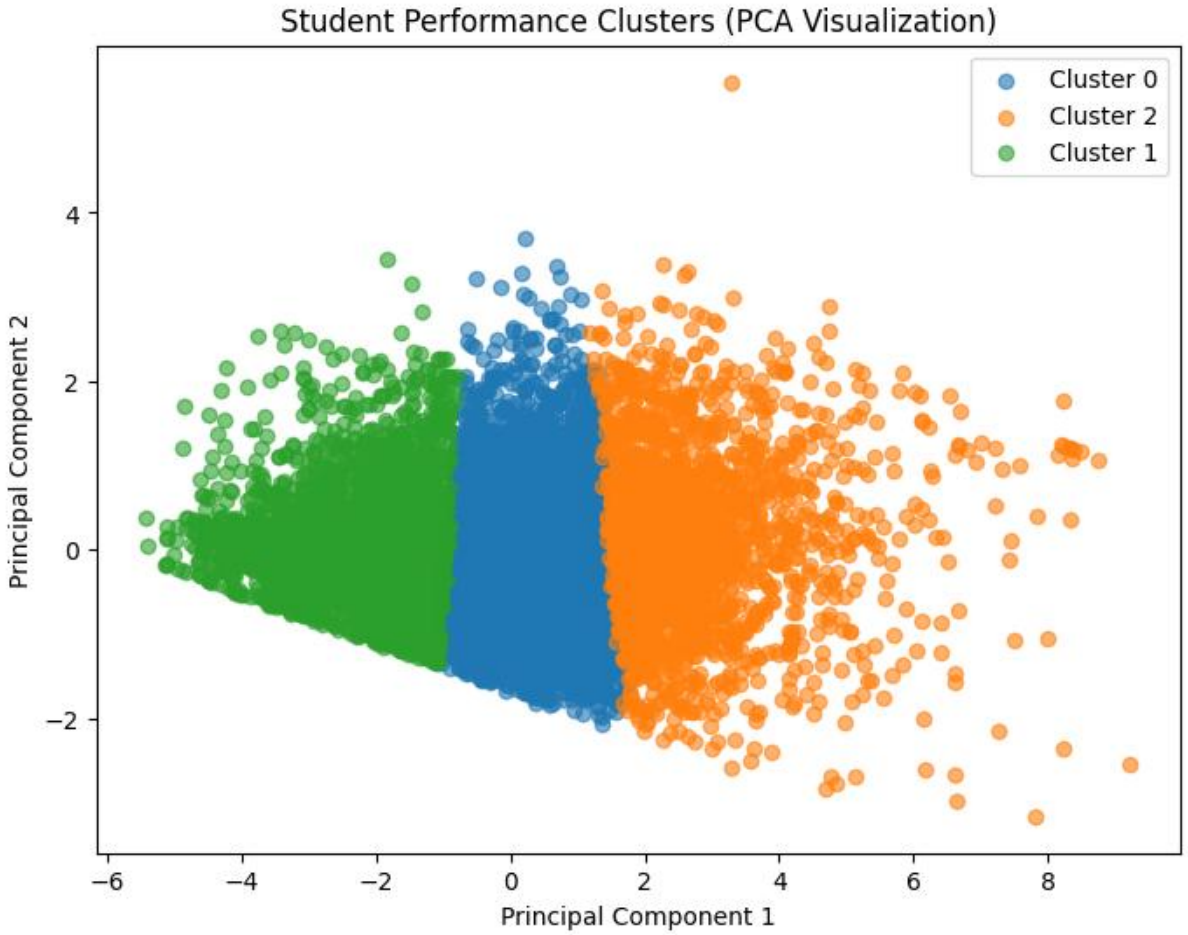
```python
performance_map = {
    0: "High Performer",
    1: "Average Performer",
    2: "Needs Improvement"
}

df["Performance_Level"] = df["Cluster"].map(performance_map)
```

12.11 PCA Visualization :

```python
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["Cluster"] = df["Cluster"]
```

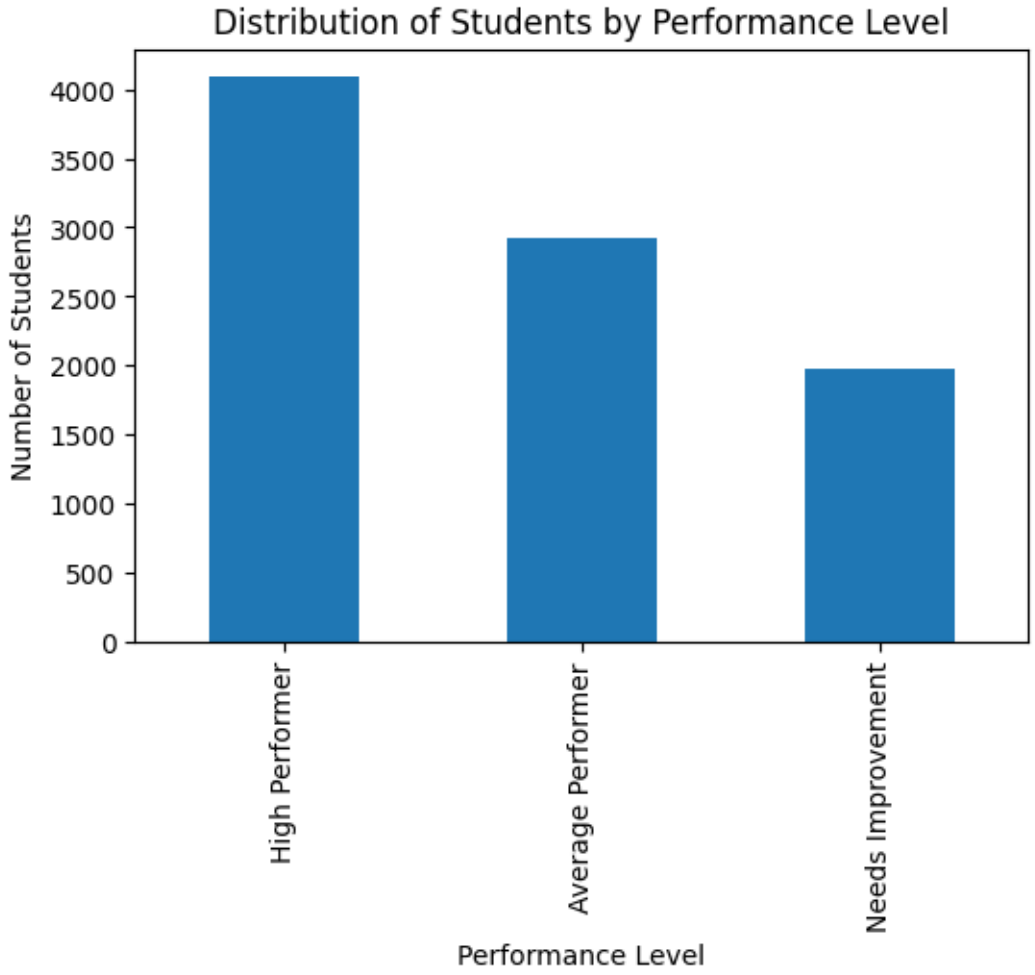
12.12 Plot Cluster Visualization :

```python
plt.figure(figsize=(8,6))

for cluster in pca_df["Cluster"].unique():
    subset = pca_df[pca_df["Cluster"] == cluster]

    plt.scatter(
        subset["PC1"],
        subset["PC2"],
        label=f"Cluster {cluster}"
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Student Performance Clusters (PCA Visualization)")
plt.legend()
plt.show()
```



12.13 Performance Distribution Chart:

```python
df["Performance_Level"].value_counts().plot(kind="bar", figsize=(6,4))

plt.xlabel("Performance Level")
plt.ylabel("Number of Students")
plt.title("Distribution of Students by Performance Level")

plt.show()
```



12.14 Evaluate Model using Silhouette Score :

```python
score = silhouette_score(X_scaled, df["Cluster"])

print("Silhouette Score:", score)
```

12.15 Save Model and Results :

```python
# Step 15: Save trained models

joblib.dump(
    kmeans,
    "D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\kmeans_model.pkl"
)

joblib.dump(
    scaler,
    "D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\scaler.pkl"
)

joblib.dump(
    pca,
    "D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\pca_model.pkl"
)

print("Models saved successfully.")
```

12.16 Student Performance Prediction Script :

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import joblib

# Load trained models
kmeans = joblib.load("D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\kmea
scaler = joblib.load("D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\scal
pca = joblib.load("D:\\VNSGU_PDA_DATA\\Data-Analytics-using-Python-Minor-Project\\models\\pca_mod

# Take student marks as input
print("Enter Student Marks:")

new_student = {
    "AWD_Total": float(input("AWD Total: ")),
    "WFS_Total": float(input("WFS Total: ")),
    ".NET_Total": float(input(".NET Total: ")),
    "LOS_Total": float(input("LOS Total: ")),
    "NT_Total": float(input("NT Total: ")),
    "CC_Total": float(input("CC Total: "))
}
```

```python
# Convert input into DataFrame
new_df = pd.DataFrame([new_student])

# Scale the input data
new_scaled = scaler.transform(new_df)

# Predict cluster
cluster = kmeans.predict(new_scaled)[0]

# Map cluster to performance label
performance_map = {
    1: "High Performer",
    0: "Average Performer",
    2: "Needs Improvement"
}

# Print prediction
print("\nPredicted Cluster:", cluster)
print("Performance Level:", performance_map[cluster])
```

```
PS D:\VNSGU_PDA_DATA\Data-Analytics-using-Python-Minor-Project\src> python predict.py
Enter Student Marks:
AWD Total: 54
WFS Total: 56
.NET Total: 43
LOS Total: 45
NT Total: 56
CC Total: 21
Performance Level: Needs Improvement
```

Chapter 13 Conclusion :

By applying different data analysis techniques and based on the statistical and visual results, we have reached the following conclusions:

1. The majority of students fall within the average academic performance range, indicating balanced overall academic results.
2. Most students scored between 55% and 75%, which represents moderate to good academic performance.
3. The pass rate is significantly higher than the fail rate, showing satisfactory academic outcomes in the dataset.
4. Network Technology (NT) has the highest number of failures, indicating that it is comparatively the most difficult subject for students.
5. Cloud Computing (CC) has the lowest number of failures, suggesting stronger student performance in this subject.
6. A strong positive correlation exists between subject marks, SGPA, and percentage, indicating that good performance in individual subjects contributes significantly to overall academic success.
7. Students who passed generally obtained higher total marks and percentages, while failing students are concentrated in the lower score ranges.
8. The K-Means clustering model successfully categorized students into three performance groups: High Performer, Average Performer, and Needs Improvement, helping to identify academic performance patterns among students

Chapter 14: References :

• Python Official Documentation
https://docs.python.org/3/

• Pandas Official Documentation
https://pandas.pydata.org/docs/

• NumPy Official Documentation
https://numpy.org/doc/

• Seaborn Documentation
https://seaborn.pydata.org/

• Matplotlib Documentation
https://matplotlib.org/stable/index.html

• Scikit-learn Documentation
https://scikit-learn.org/stable/

• VNSGU Student Result Portal (Dataset Source)
https://ums.vnsgu.net/Result/StudentResultDisplay.aspx?HtmlURL=7596,0000

• UCI Machine Learning Repository
https://archive.ics.uci.edu/

• GeeksforGeeks – Machine Learning Tutorials
https://www.geeksforgeeks.org/machine-learning/